

Vulcan Activity Tracker

User Manual



Team Members:

Margo Bonal

John Gerega

Luke Ruffing

Professor: Dr. Weifeng Chen

Course: CMSC-4920-Senior Project II

School: Pennsylvania Western University

Instructor Comments/Evaluation

Table of Contents

Section 1:	4
Project Title: Vulcan Activity Tracker	5
Project Overview:	5
Motivation:	5
Comparison:	6
Community or Social Implications	6
System Block Diagram	7
Project Implementation Details	8
<i>Difference from Design Document</i>	8
<i>Challenges During Implementation:</i>	11
Software Engineering Principles:	13
Section 2:	15
User Manual Description:	16
User Instructions:	16
<i>Welcome Page:</i>	16
<i>Account creation:</i>	17
<i>Login:</i>	18
<i>Dashboard:</i>	18
<i>Training Page:</i>	20
Maps Page:	22
Clubs Page:	23
Teams Page:	25
Leaderboard:	28
Challenges:	29
Settings:	31
Section 3:	34
Developer Instructions:	35
<i>Step 1:</i>	35

<i>Step 2:</i>	35
<i>Step 3:</i>	37
<i>Step 4:</i>	38
Team Contact Information:	40
List of References	41
Table of Index / Glossary	42
Appendix I: Writing Center report	46
Appendix II: Team Details	47
Appendix III: Workflow Authentication	48
Appendix III: Code listing	49

Section 1:
Project Overview

Project Title: Vulcan Activity Tracker

Project Overview:

The Vulcan Activity Tracker is a fitness app designed for logging and sharing athletic activities with friends. It incorporates social interaction and personal goal tracking for campus students. Our digital athletic logbook offers several features for planning routes, monitoring progress, and competing, for running, cycling, swimming, and many other sports. Our app is designed to push Vulcan athletes to train at their highest performance, while encouraging healthy habits for casual users. Encrypted account creation and login portal ensures that student data is secure, demonstrating our utmost priority of user safety. Modern user-friendly dashboards allow easy use. Activity history can be stored and viewed at any time with dynamic searching. Clubs can be created and managed. Challenges can be created by users and are available to platform users. These can be anywhere from daily step goals, summer training plans, or weightlifting PRs. Full customization makes the Vulcan Activity Tracker flexible for each student's needs! Leaderboard features additionally allow platform users to interact with each other in friendly competition. Earning a spot on the weekly leaderboard motivates users to get outside or into the gym! The Vulcan Activity Tracker is free and available for all students with valid PennWest California credentials, regardless of if they are actively on an official campus team, intramural club, or individual.

Motivation:

Accessibility and usability were the founding principles and motivation for the Vulcan Activity Tracker, ensuring that individuals of all abilities can easily track, understand, and improve their daily activities without unnecessary complexity. The Vulcan Activity Tracker is

intended to serve as an athletic activity management system specifically for PennWest California students. Students serve as the intended audience; however, campus faculty such as coaches and professors can access as well. Allowing students with strict budgets to have access to fitness tracking while eliminating hefty subscription that locks these valuable features, motivated our development team to produce this senior project deliverable. We experienced this paywall issue firsthand with fitness tracker applications such as *Strava (2026)* and aspired to solve this problem.

Comparison:

The question arises: there are many current athletic activity trackers available; what makes The Vulcan Activity Tracker different? Apps on the market right now are activity specific, alienating both uncommon and unpopular exercises from using the app features. For example, *Nike Run Club App (2025)*, *Runna (2025)*, and *Strava (2025)* are directed towards running athletes and users only. Whereas, *Strong (2025)* and *Hevy (2025)* are designed for weightlifting activities. While many other activity trackers may provide exclusive features and smart watch connectivity, they are costly and detrimental to a student's overall budget. In one combined platform, the Vulcan Activity Tracker aspires to meet student organization needs, social interaction, and exercise goal planning in a free accessible way.

Community or Social Implications

The purpose of the Vulcan Activity Tracker is to bring together the PennWest student population into one athletic app that allows for the tracking of all campus sports, cycling, weightlifting, calisthenics, swimming, running, climbing, equestrian sports, and general exercise categories. Building a community base on campus not only helps students feel motivated by

others but also allows students to have a break from academic stress by participating in fun activities with others that share similar interests.

System Block Diagram

The Vulcan Activity Tracker consists of 2 main software components, Client (Frontend) and Server (Backend) Sides. Client Side will be user interface that is accessible by internet hosting. Server-Side drives functionality of the application that will be designed to handle data propagation and database storage. The UI prepares and sends fitness activity data to the Backend Server. The Server then cleans the data and prepares it for database entry. When received by database handling logic, the fitness activity data is then sent into and stored by the MSSQL database. Below, this modular process is depicted in the system block diagram.

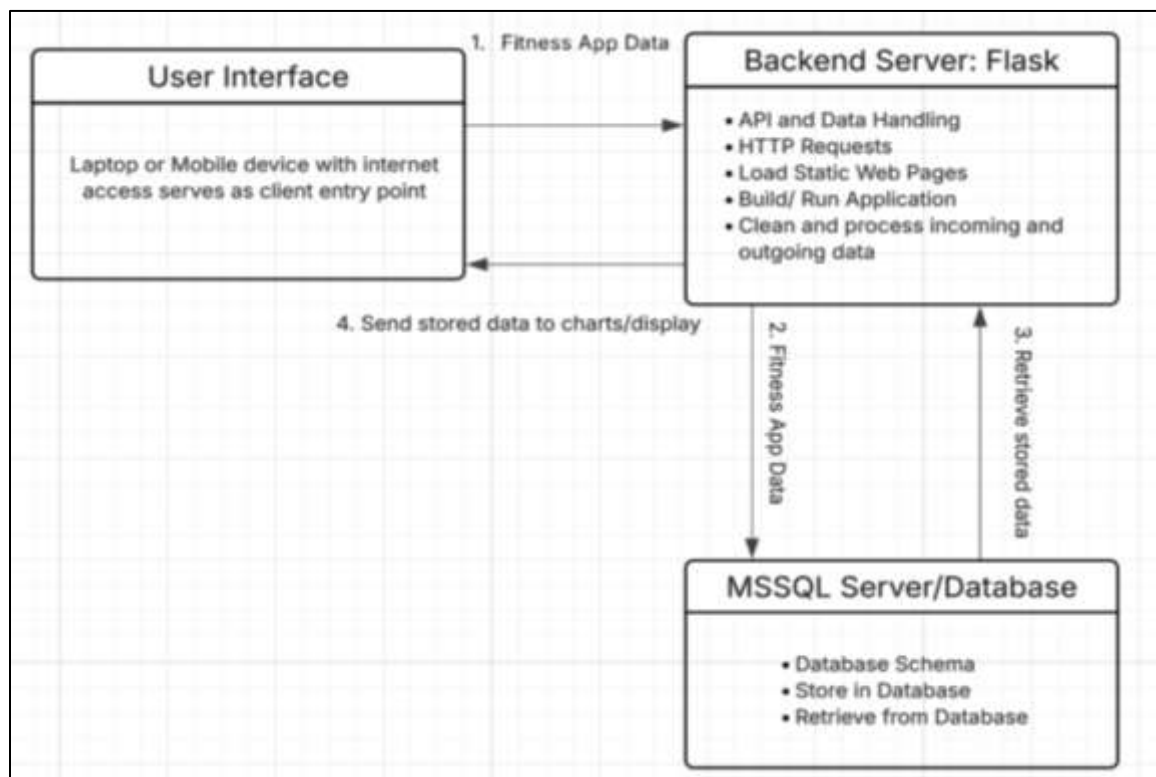


Figure 0: System Block Diagram

Project Implementation Details

Difference from Design Document

During the initial stage of this project, our development team drafted a Design Document to set overall goals for the Vulcan Activity Tracker. Within this design document, the modular setup, server details, client user-facing features, and data-flow processes were discussed. To provide project verification, the user manual will now compare initial stage and post-delivery maintenance versions to ensure consistency.

Module Cohesion was originally defined as the following:

- The Login & Authentication Module: This module focuses solely on validating the login credentials of users.
- The Activity Management Module: This module handles only the creation, validation, removal, and retrieval of activities.
- The Group Management Module: This module handles only the management of club/group memberships and roles.
- The Leaderboard Module: This module focuses only on processing and displaying the rankings within different parts of the Vulcan Activity Tracker.
- The Database Module: This module solely manages the storage and retrieval of data items into and out of the database.

Module Coupling was originally defined as the following:

- The Login & Authentication Module: This module interacts with the Database Module to verify the user's login credentials.

- The Activity Management Module: This module interacts with the Database Module to store and retrieve activity data.
- The Group Management Module: This module interacts with the Database Module to store and retrieve group data.
- The Leaderboard Module: This module interacts with the Database Module to store and retrieve leaderboard data.
- The Database Module: This module interacts with all the previous modules to send data back and forth to the database server.

Both Module cohesion and coupling were achieved following the design document roadmap. Modularization kept development organized. The database schema was created based on these components. However, the modularization differs since our team had initially proposed a smaller scale than the one that resulted. The Vulcan Activity Tracker's database architecture became quite involved, handling both encrypted user data and recurring site information. Classes and objects were also followed from the design document.

Physical file storage remained a system vulnerability and was out of scope for the Vulcan Activity Tracker. All data was stored in the SQL database. This avoided the vulnerabilities of locally or session-stored data. Additionally, avoided the complex logic of file validation of uploading or downloading files to or from the system. SQL Server access provided timely data lookup and table propagation for both login and application features.

Real-time requirements were correctly projected in the Design document. WIFI connection was for internet access without a cellular data service provider or for device access without SIM connectivity. Efficiency was also reached, as it was noted as a concern for platform response

time. This can be seen in loading static interface pages into the server, reducing rendering time.

The concern of potential losses of internet connection and having negative effects on the completion of data storage remains a problem.

The proposed PDL Outline in the Design Document was successfully achieved; however, the Vulcan Activity Tracker was expanded with many components that were not initially specified.

Original Outline:

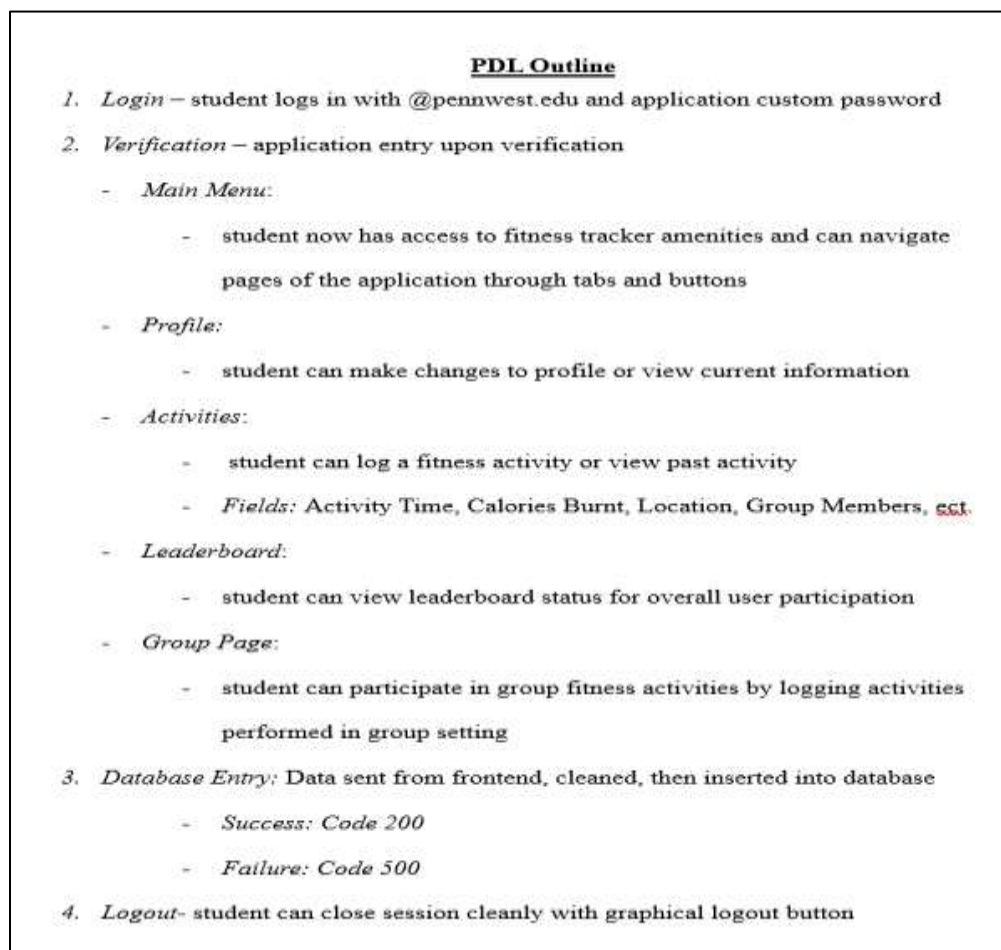


Figure 1: Original drafted PDL Outline

Difference: Addition of Settings, Challenges, Teams, Dashboard, Inbox, Welcome pages, along with the resulting server configuration and schema architecture.

Programming language and database management systems were kept consistent throughout this project. Python was originally projected in the design document and was maintained throughout. Same consistency with Microsoft SQL Server database systems. All software tools were achieved through Open-Source services. However, features that were not initially proposed in the design document are the Map viewer APIs and Weather forecast APIs. These two features were researched and outsourced since creating our own was outside the scope of this project. These APIs were Map API Leaflet (2025) and OpenWeather (2026).

Finally, the projected Implementation timeline was followed and met. January through April were reserved for primary development time. April through May was set for project validation, revisions, testing, and final product presentation. The design document served as a contractual agreement for a defined timeline, and now, in post-delivery maintenance, it can be confirmed.

Challenges During Implementation:

Over the course of the semester, the development of the Vulcan Activity Tracker faced several technical and logistical challenges that tested the team's problem-solving abilities. A major challenge arose with database compatibility. The team discovered that MSSQL Server 2022 (version 17) was incompatible with ARM 64 architecture, as it was designed for Intel processors. Potential solutions, including the use of virtual machines or acquiring Intel-based laptops, proved to be financially and logistically unfeasible given the project's time and resource constraints.

An issue encountered was with the leaderboard functionality. While the table successfully extracted sport-specific data, it failed to rank users correctly. Initially, the leaderboard simply

reflected the order of entries in the database rather than actual totals, which undermined the intended competitive functionality and required a reassessment of the ranking logic.

The activity tracking table also presented complications due to timing issues. When users submitted activity forms, the updates were not immediately reflected in the viewing table, requiring manual page refreshes. This behavior broke the user's perception of a seamless, real-time experience. Investigation revealed that asynchronous JavaScript functions were not paired correctly with await statements, and logic errors occasionally prevented the data from being retrieved in time.

Other functional issues included radio buttons, which were initially implemented but lacked the necessary logic to operate correctly. Additionally, email functionality became a challenge when attempting to implement verification and password recovery features. Although a frontend page was successfully created and the Resend email service integrated, backend errors prevented complete functionality, requiring ongoing debugging and further refinement.

Feature design decisions also created challenges. The team debated whether users should be able to add badge images to challenges, which led to broader questions about supporting profile picture uploads. Finally, the development of the Teams page proved particularly difficult. Verification of user roles, whether coach or team member, was not functioning, preventing access to the page and hindering backend integration. Moreover, the database currently contains empty tables for teams and members, making it impossible to fully test or display content.

Overall, these challenges highlighted the complexity of coordinating multiple systems, technologies, and features within a single project. Despite the obstacles, our team developed

creative solutions, gained valuable troubleshooting experience, and identified key areas for improvement in future iterations of the application.

Software Engineering Principles:

During the Fall 2025 and Spring 2026 senior semesters, our development team used foundational software engineering principles to guide project initialization through completion. Agile Software Development Lifecycle was the method that met both team and project requirements. The Agile lifecycle allows for changes during development, while using documentation as a guide. We identified that Linear development would not allow for the Vulcan Activity Tracker to grow and shift during the process. Additionally, Agile uses incremental development, increased adaptability, and fast component creation when dealing with short timelines. The final feature of Agile that aided in our project's development was the method of splitting tasks between team members, which accelerated project development.

However, the documentation stages of the Waterfall software development lifecycle were implemented to record requirements and goals. These stages are requirements, analysis, design, implementation, post-delivery maintenance, and retirement. Four technical documents were prepared, drafted, and revised to certify the Vulcan Activity Tracker's creation. These documents were the initial Proposal, Requirements, Design, and Specifications. A Gantt Chart was used in correspondence to align the team schedule along with semester milestones and due dates. All these documents additionally serve as binding contractual agreements between team developers and projected goals.

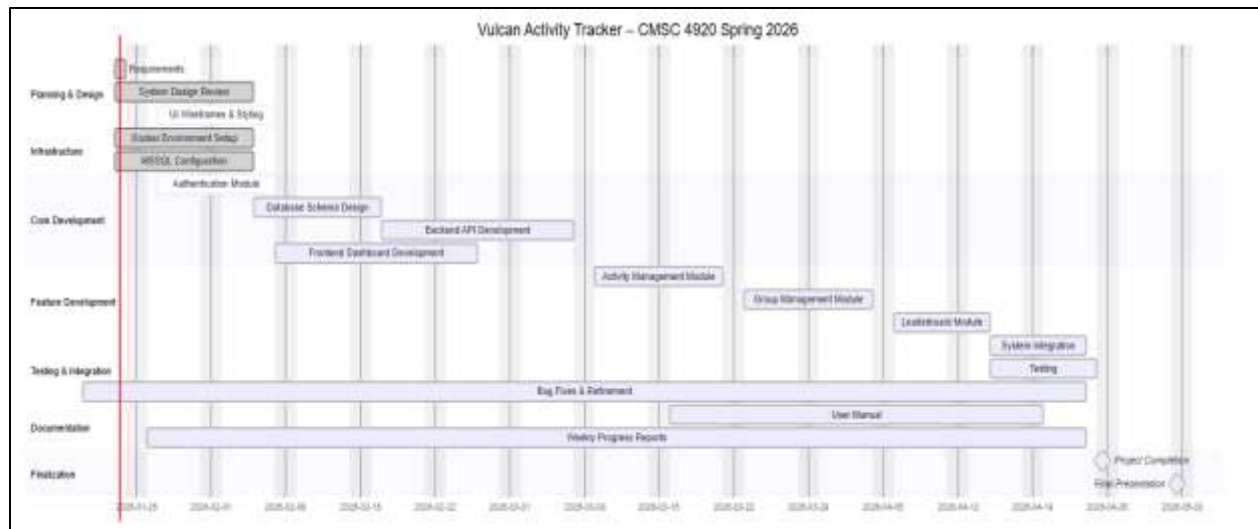


Figure 2: Gantt Chart

Section 2:

User's Manual

User Manual Description:

This document serves the purpose of a user manual for the Vulcan Activity Tracker Senior project 2026. Prepared by the development team, this user manual represents both instructions for developers and users. The user case will demonstrate where Pennsylvania student users can find and access the platform. Additionally, instructions to interact with platform features will be included. On the software side, a step-by-step guide will allow developers to build and access the platform locally on their machines.

User Instructions:

These instructions are intended for Pennsylvania Western University, California campus student users. Below are step-by-step instructions for finding the Vulcan Activity Tracker online by domain, account registration, and feature description. The overall goal of this document is to teach users to fully manage and interact with their app accounts. Additionally, this user manual will enable users to understand how their information is viewed (public or private) and what can be either shared or hidden from other users. Our goal as the development team when creating this project was to make sure that all students could use our app regardless of qualifications or specific sport. The Vulcan Activity Tracker provides important daily habit tracking that is otherwise blocked by a paywall in other similar applications. Below, users can find all the information needed to fully enjoy the Vulcan Activity Tracker's features. While there is a designated developer section of this document, developers are encouraged to read the user instructions to understand user interface components.

Welcome Page:

Visit: <https://vulcanactivitytracker.onrender.com>

The welcome page demonstrates features and overall information of the Vulcan Activity Tracker. Training environments are displayed on the welcome page, however, more information can be found within the application once an account is created. By navigating to Log In, users can create an account to enter the app and access fitness tracking features.



Figure 3: App Welcome Page

Account creation:

Users can create an account by navigating to the Create Account page. By entering a valid Pennwest.edu email address, a username, and a password, Students can successfully make an account. Password requirements are displayed on the form to show length, special character, and uppercase requirements.

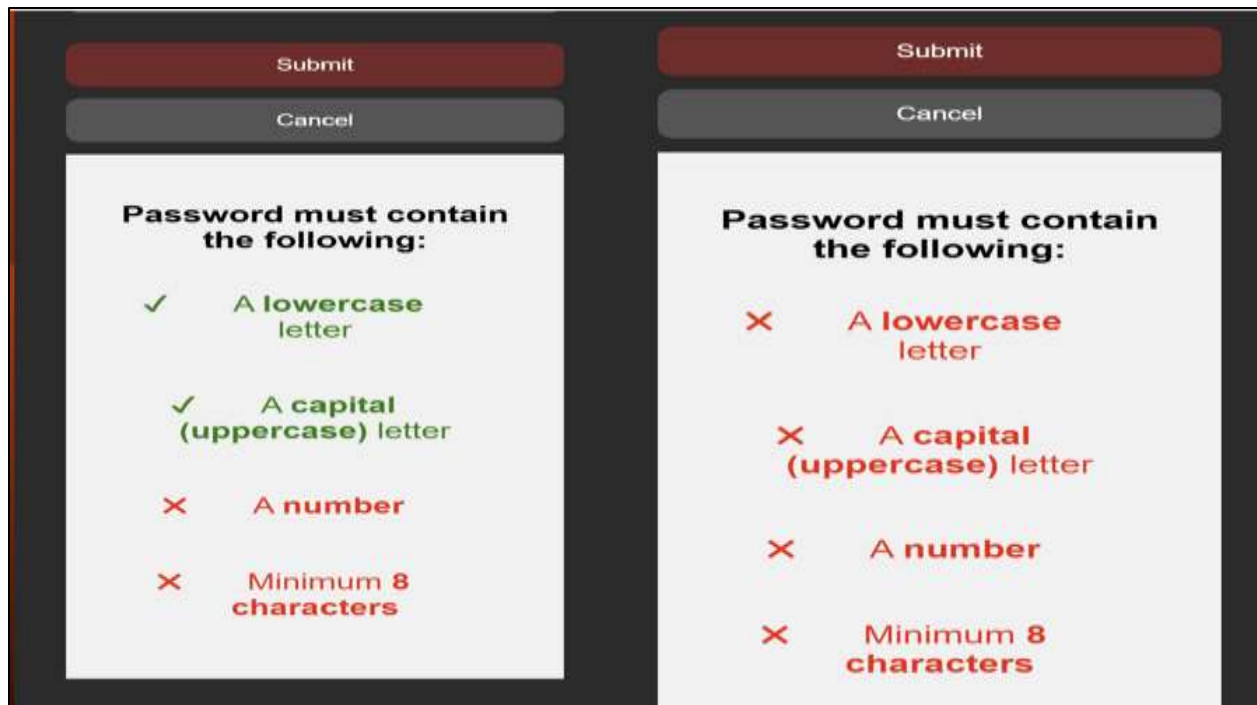


Figure 4 : Password Requirements

Login:

Only after the account is successfully created can a student enter the Vulcan Activity Tracker. To log in, students must enter their unique username and password.

Dashboard:

The following diagram displays the Vulcan Activity Tracker's main dashboard. Here, users can view the top ten activity overview. Users can view what challenges they are participating in. Joined clubs can be seen from the dashboard as well. Users can browse a list of available friends and follow their friends to keep updated on what activities and clubs they are in. The main profile card keeps count of quick stats of clubs joined, logged activities, and account followers. Each activity shows a like count as well. Friends can only view each other's public activities, providing an option for users to set certain activities to private. While accounts remain

public, this feature enables users to participate in the way they feel comfortable. The Vulcan Activity Tracker promotes positivity; therefore, there is no dislike button.

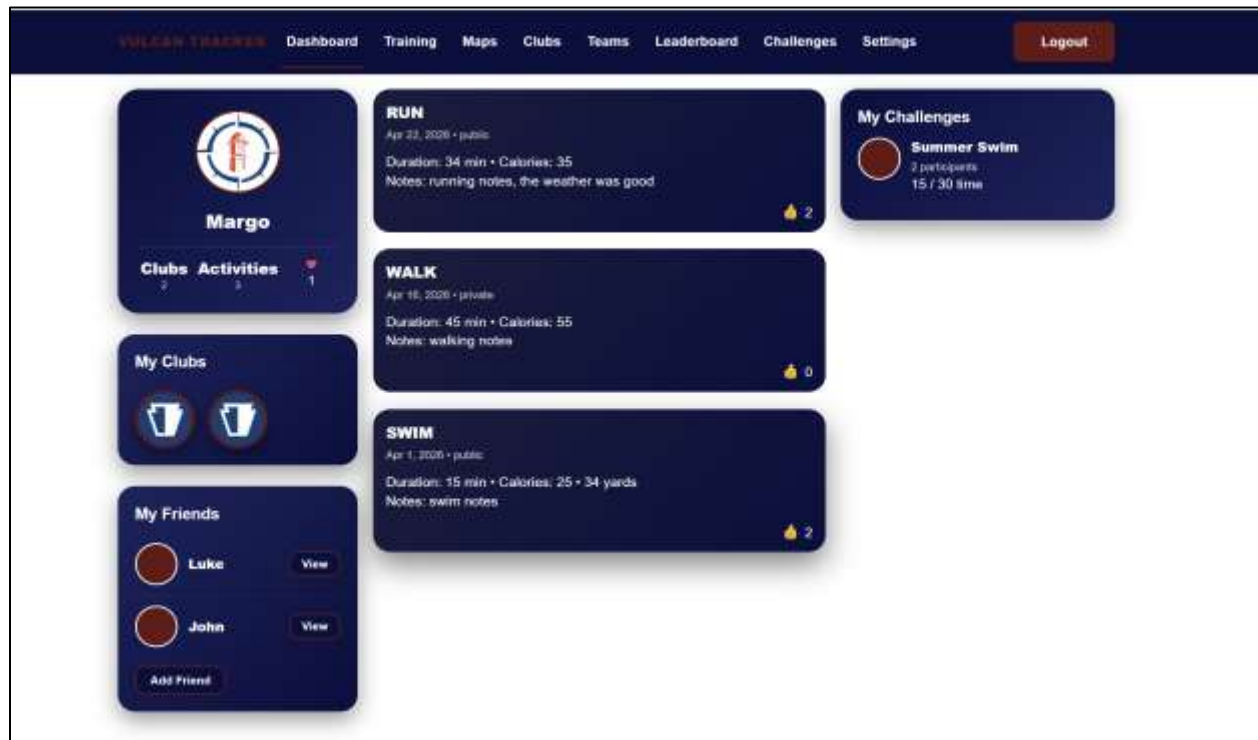


Figure 5: Dashboard

Additionally, the friends feature can be accessed through the dashboard. By clicking *View* next to a friend's profile, the Friend Portal will pop up. Since the friends feature acts as an overlay, users can feel like they are still within their own dashboard rather than moving to an external feature. Public information is displayed here.



Figure 6: Friends Feature Modal

From the dashboard, each page is accessible for users to navigate to from the top navigation bar. This bar stays visible no matter what page is being viewed. This enables users to have easy access to each feature.

Training Page:

The next feature of the Vulcan Activity Tracker is the training page. This page serves as a hub for all athletic and fitness activities. Here, workout data can be entered by category/activity type. Appropriate metric fields are accessible upon selection. In the figure below, the activity form defaults to fields that are common between all sports. Date, duration, Calories, and Accessibility. However, once a user selects a sport from the Activity Selection dropdown menu, the activity form will be customized with sport-specific fields. This dynamic method allows users to enter specific details on the activity they participated in.

Figure 5 shows the activity Selection dropdown menu with several activity types for all athletes. When an activity is selected, the display badge at the top of the form will reflect what type has been selected. This allows users to be reminded of selected activity types when filling in the form. The activity form can also be cleared at any time during the data entry process,

resulting in empty form so users can start form over if a mistake was made or no longer want the information.

Additionally, in figure 5, activity history can be searched for easy viewing. The search bar filters by name, date, calories, public, private, notes. Each of these filters can be used as a keyword; however, users should note that the search bar is case sensitive.

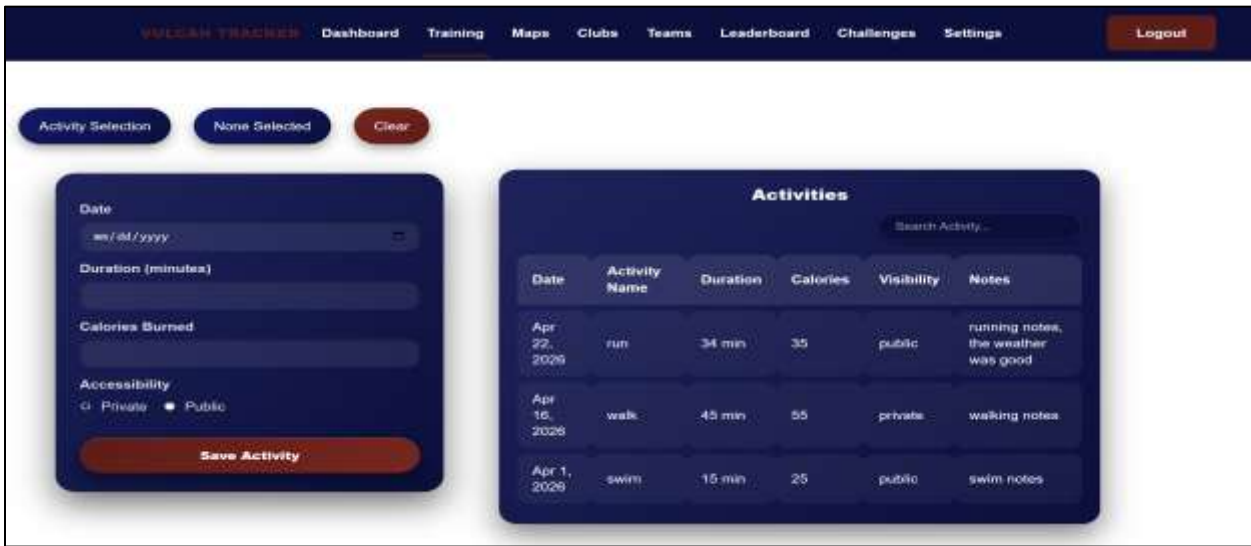


Figure 7: Training Page



Figure 8: Activity Selection and Search bar

Maps Page:

The third page of the Vulcan Activity Tracker holds many customizable features for users to interact with! Users can create their very own new running, walking, or cycling routes. By clicking *Start Route*, users can draw on the map, placing pins and map markers as they go. When the new route has been completed, enter a name for the route and click *End Route*. This signals the route to be stored in the user's account for viewing again. A marker will always remain in the center of campus to aid in relocation when the map is zoomed out.

Routes can be viewed by clicking on routes stored under My Routes. They will be redrawn on the map for viewing in red. Routes can be deleted when hovered over, and the distance in miles can be seen. Vulcan Routes holds official campus running routes that are available in the California campus gym. Clicking on these routes displays them on the map in blue.

The final feature of the Maps page is a live weather forecast. This provides a weekly weather forecast view. Recommendations are made in yellow based on optimal athletic conditions, i.e., days with the lowest chance of rain, wind, snow, etc...

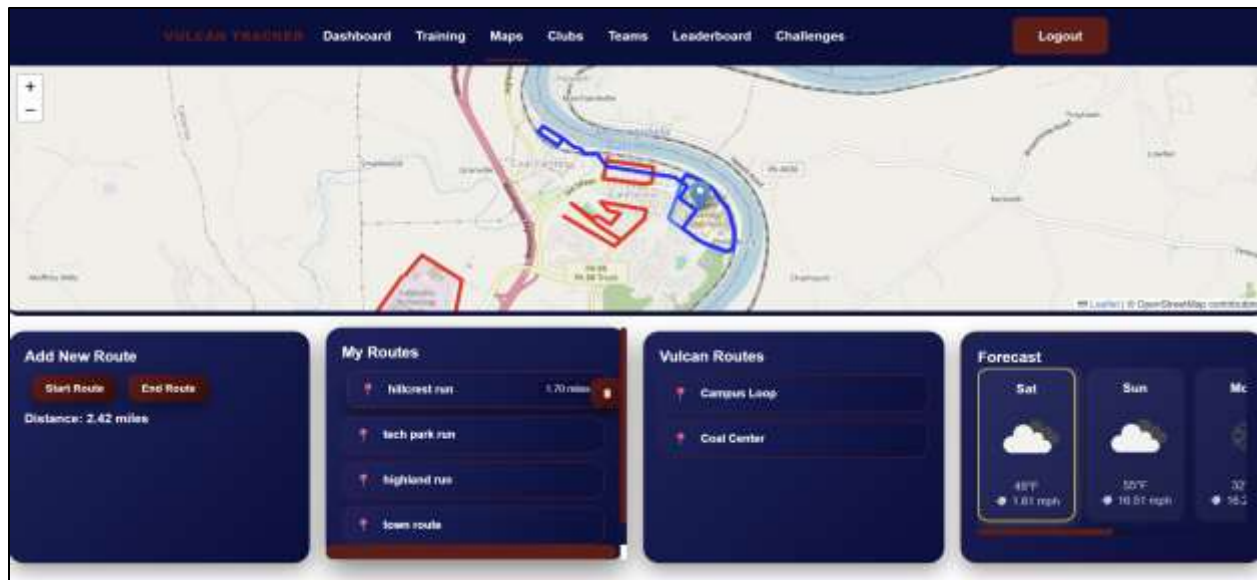


Figure 9: Maps View and Route Builder

Clubs Page:

The clubs page of the Vulcan Activity Tracker allows users to join clubs that can be made for recreational groups, campus-wide organizations, or even club sports teams. This page is a hub where users can navigate to discover clubs, join clubs, create clubs, and interact with clubs. After navigating to the club's page, the user is presented with the creation of a club button and two primary sections: My Clubs and Available Clubs

My Clubs displays all the clubs that the current user is a member of or the owner of. In this section, a user can quickly access their clubs without having to search through a long list. The club cards in the My Clubs section have two buttons: view, which brings the user to the club's details page, and leave/delete, which removes the user from the club if they are a member and deletes the club if they are the owner.

Available Clubs display all the clubs that have been created by other users. Each club has a card that shows the club's name, description, number of members, and whether the club is

public or private. In this section, a user can scroll through the different clubs to find one that interests them. Like the club cards in the My Clubs section, there are two buttons: view, which brings the user to the club's details page, and join/request to join, which will display just join if the club is public and will add the user to the club. If the club is a private club, it will display a request to join, and when clicked, it will send a request to join to the owner of the club's inbox.

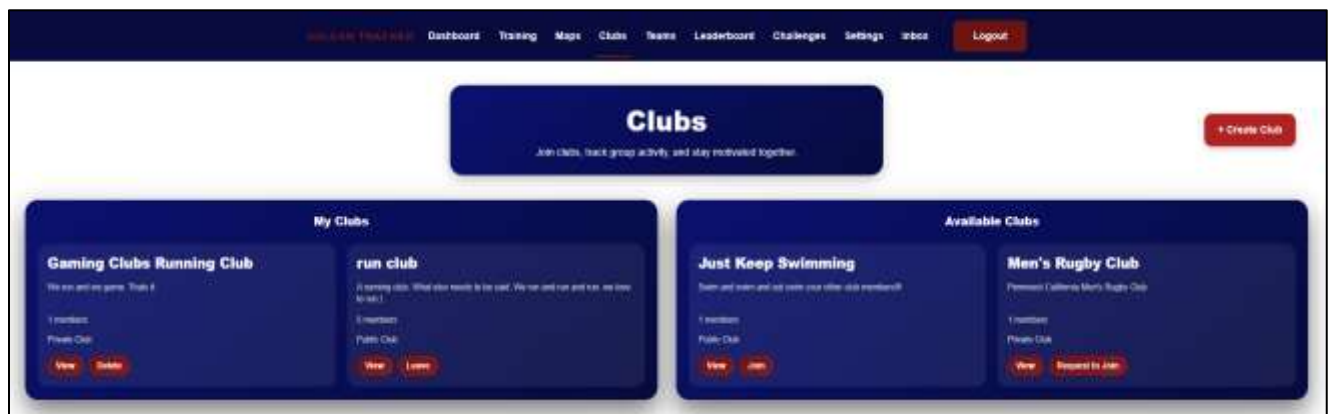


Figure 10: Clubs Page

On each club's card, there is a view button that navigates users to a club's details page. Each club's details page is unique, showing the club's name, description, club leaderboard, members' recent activities, and member list. Along with that, there is also a small card that contains the club's info, such as member count, sport type, and privacy.

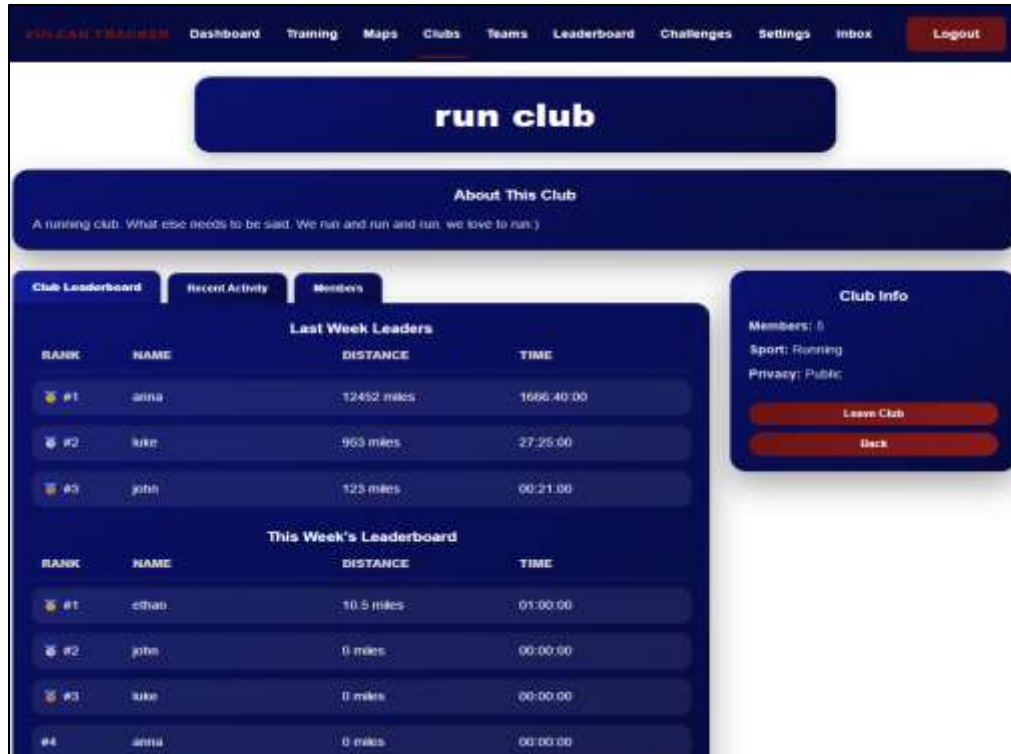


Figure 11: Clubs Details Page

Teams Page:

The teams page feature enables users who are a part of one of the university's NCAA sports teams. This feature is limited to those who are part of these teams: coaches and players. A regular user cannot access a team-specific page unless they are invited by the coach. Coaches can invite usernames, create announcements, and create events like practice or games. The user will receive the invite in their inbox and can decide to accept or deny the invitation to the team. Once a user is invited and has joined the team, they can view announcements and events but cannot

create them. A mini leaderboard is also on the teams page, containing only those who are on the roster of that specific team.

The figures below show the team's page from two different views. Figures 12a and 12b show the team's page from the coach's point of view. As shown, the coach has many more privileges available to them. Figures 12c and 12d show the teams page view from an athlete's point of view



The image shows a dark-themed user interface for a coaches' team page. It features two main sections: an announcement form at the top and an event creation form below it. The announcement form includes a title field, a large text area, and a 'Post Announcement' button. The event creation form includes fields for event title, type (a dropdown menu), date (with a calendar icon), location, and extra notes (a large text area), followed by a 'Create Event' button.

Announcement title

Write announcement here...

Post Announcement

Event title

Select type

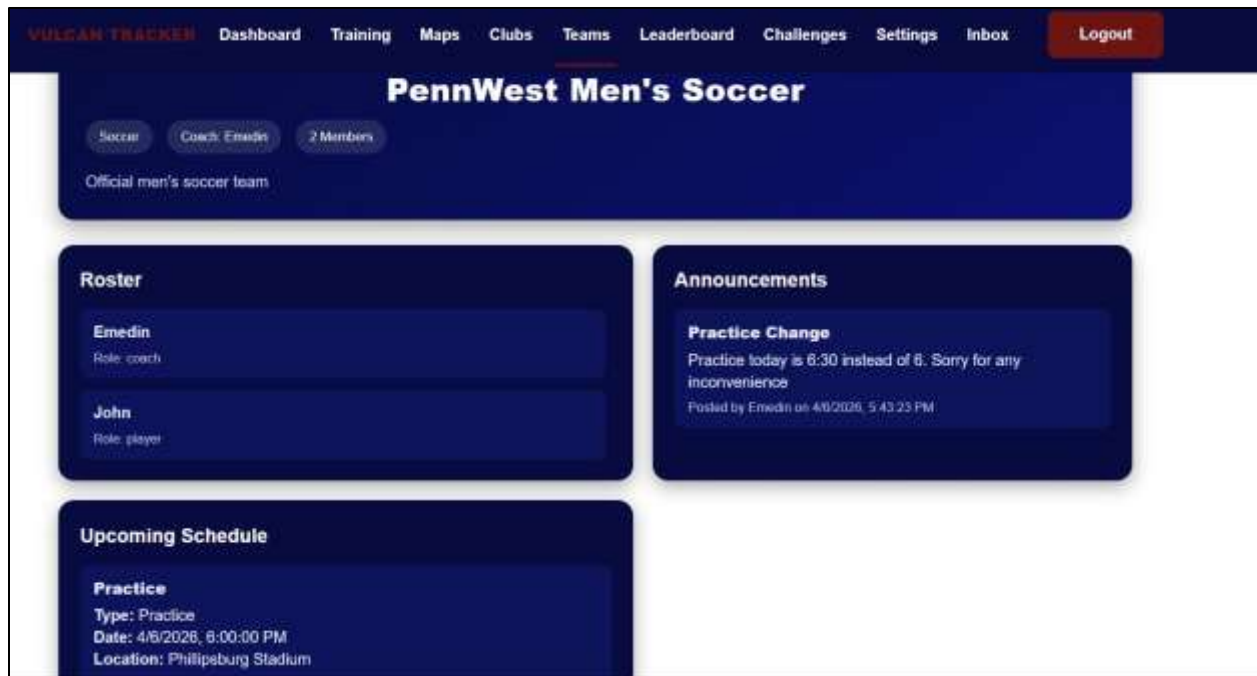
mm/dd/yyyy --:--

Location

Extra notes...

Create Event

Figure 12a (top) and 12b (bottom): Coaches team page view



Team Leaderboard

Running Score Desc

Rank	Name	Miles Run	Total Duration (minutes)	Total Activities
1	John	3.11	29	1
2	Emedin	0	0	0

Figure 12c (Top) and 12d (bottom): User view of the team page

Leaderboard:

The leaderboard feature enables users to participate with each other in friendly competition. This feature allows users to filter by sport type, score, duration, and distance. The leaderboard can also be seen in ascending or descending order. The top 3 winners will be ranked by gold, silver, and bronze medals. The Vulcan Activity Tracker leaderboard is updated

automatically based on user accounts. Users should check back periodically to see updated statistics for the sports they participate in.

Rank	Name	Score	Total Duration (minutes)	Total Activities
1	Margo	118	118	3
2	John	45	45	1
3	Luke	43	43	1

Figure 13: Vulcan Leaderboard

Challenges:

The challenges page enables Vulcan student users to create fun athletic competitions. These can be private, where a user records data and challenges themselves, or they can be sent to public, where all Vulcan Activity Tracker users can participate. A challenge can be created by clicking Create Challenge and filling out the challenge entry form. After completing and clicking submit, the challenge is then active and stored for future use.

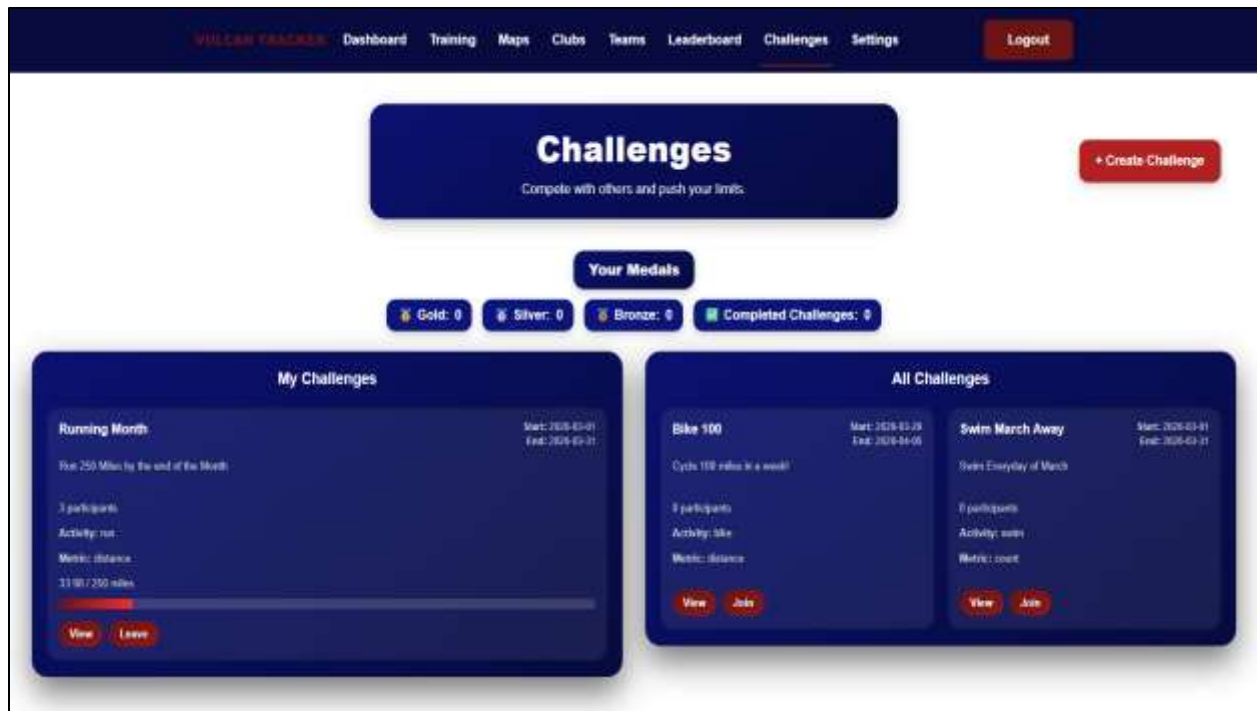


Figure 14: Challenge Page

Within the challenge feature, users can select to view the challenge or leave the challenge. When a view is selected, an overview of that challenge, participants, and a mini leaderboard can be seen. The top 3 ranks on the leader board are signaled with gold, silver, and bronze medals. Progress bars demonstrate how far along the challenge users are, while start and end dates are represented at the top of each. The left-hand side shows challenges user created or have joined, whereas the right-hand side shows challenges that are available to join made by other users. Students should be aware that if they join another user's challenge, their data will be public to others.



Figure 15: Challenge View and leaderboard

Settings:

On the settings page, users can view quick information regarding their account. This information is currently logged in the username, email associated with the account, and if their status is active. The Quick Stats tab shows overall activity numbers, clubs joined, and challenge goals set. Username can be changed in the settings feature. However, users must log out after they change their username to allow the platform to reset. Accounts can be deleted from the settings as well.

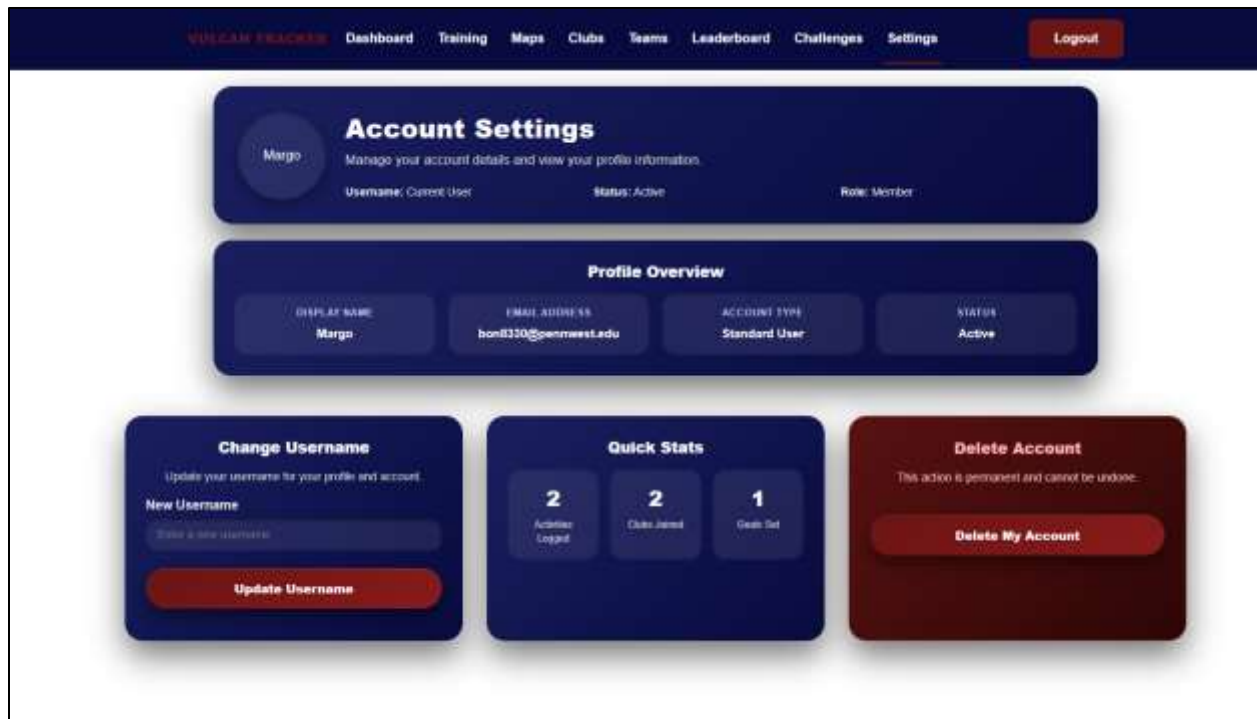


Figure 16: Setting page Features

Inbox:

The user inbox is designed for users to accept/decline invites from team coaches. Users will see a red dot next to their inbox when they have an unread invite that they still need to view. Once they go to their inbox, the invites will be marked as read. The user then has the option to accept or decline invites. If they accept, the team is updated to include their name in their roster, and the invite is cleared from the inbox. If they decline, the invite clears from the inbox with no updates.

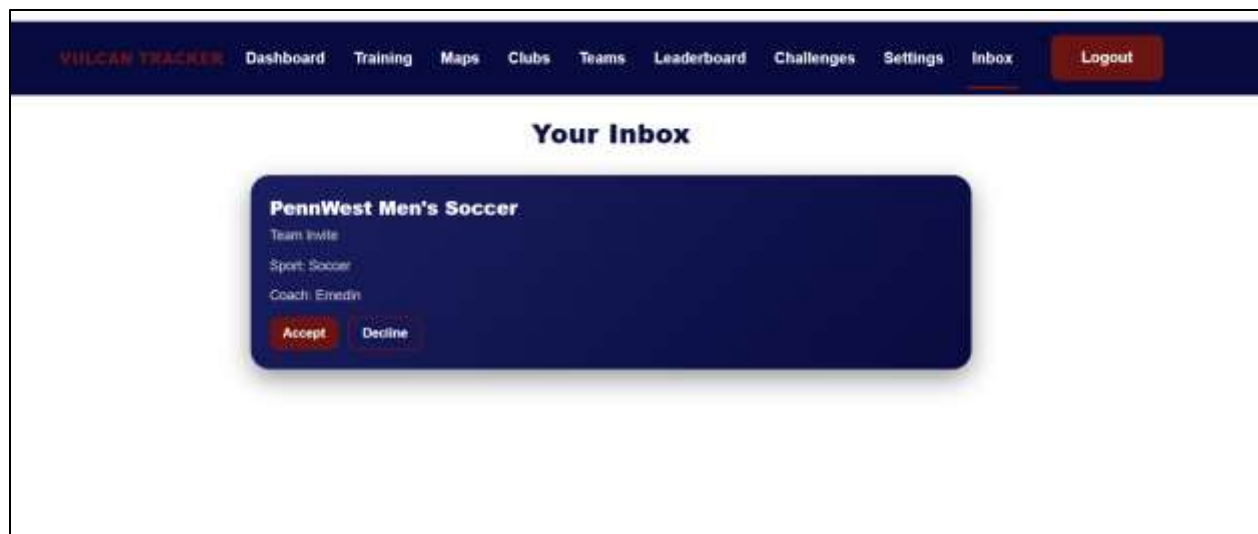


Figure 17: Inbox

Section 3:

Developer Manual

Developer Instructions:

The following developer technical instructions are for Windows 10/11 operating systems. To build and run the Vulcan Activity Tracker codebase locally from developers' machine (laptop or PC). Source code solution will be submitted upon senior project deadline. This will include a ZIP folder of the project code base, named Vulcan Activity Tracker.

Step 1:

The first step will be to download the Vulcan Activity Tracker folder from D2I. Move the folder to a location on computer where it is easily accessible and isolated from other software that may interact or cause disturbance. Our development team recommends either the Documents folder or an empty OneDrive Folder. Once the file is placed, unzip the folder or extract the file contents. The files are then prepared for the next step.

If in any case the ZIP file is unaccessible, the folder can be found online at our group GitHub repository: <https://github.com/jjdeuce06/VulcanActivityTracker>

Here, the Vulcan Activity Tracker can be downloaded manually, or can be cloned/ssh directly into code editor.

Step 2:

Developers must prepare their computer to build the Vulcan Activity Tracker codebase. Since containerized, Docker, an external containerization software must be installed. Navigate to the site: <https://www.docker.com/> and proceed to download Docker Desktop. Select your proper operating system, in this case Windows.

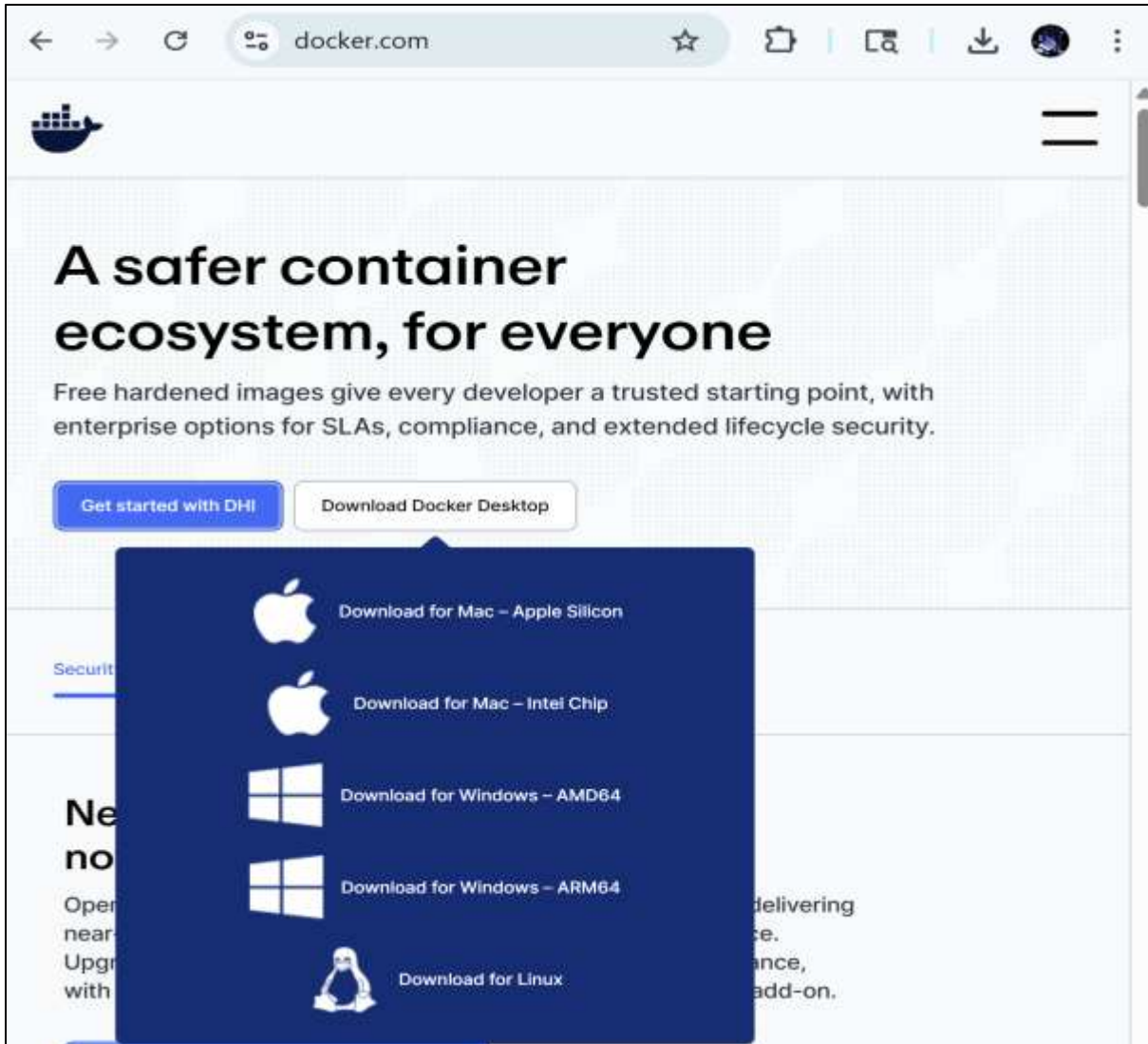


Figure 18: Docker Desktop Download

Proceed through the installation process. Docker Installation Wizard will provide additional steps to ensure properly installed. After installation, the developer will be prompted to restart computer. When system is rebooted, Docker Desktop will be ready to use. Installation success can be verified in terminal with command: `docker --version` with expected output as Docker version 24.0.x followed by a build number.

Step 3:

Now navigate back to the folder where the Vulcan Activity Tracker is stored. In either a code editor like Visual Studio Code, or right from the command prompt. The app can only be run if the environment is in the Source Code File. Move through files in the terminal with the `cd` command along with the folder name, i.e., `cd SourceCode/`

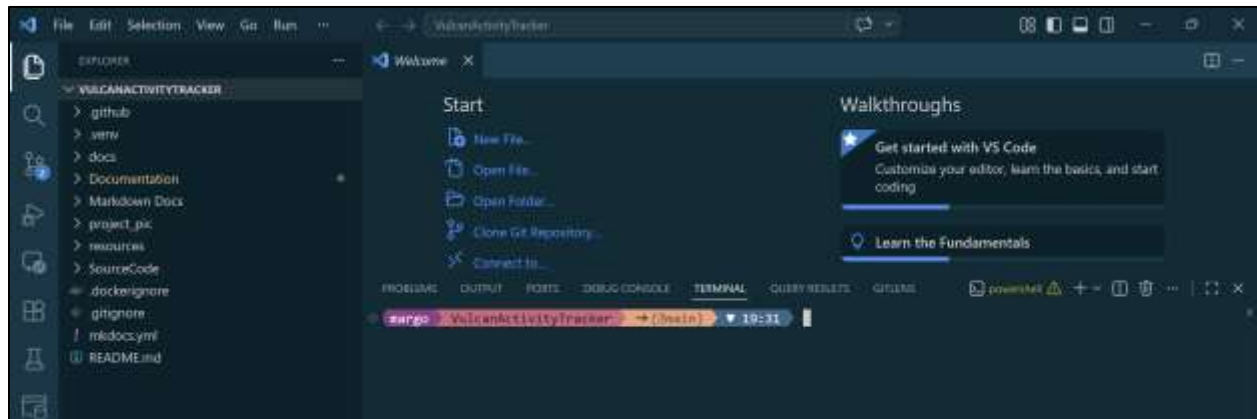


Figure 19: Vulcan Activity Tracker Open in code editor

Within the source code file, all files to properly build the project will be accessible.

Mode	LastWriteTime		Length	Name
----	-----		-----	----
d----	2/28/2026	6:52 AM		.devcontainer
d----	1/12/2026	12:51 PM		server
d----	1/12/2026	12:51 PM		templates
-a----	3/9/2026	7:01 PM	224	.env
-a----	2/28/2026	6:52 AM	11	.gitignore
-a----	3/21/2026	10:47 AM	1764	app.py
-a----	3/21/2026	10:47 AM	1028	docker-compose.yml
-a----	3/21/2026	10:47 AM	823	Dockerfile
-a----	3/21/2026	10:47 AM	113	requirements.txt

Figure 20: Source code folder contents

Step 4:

The fourth developer technical step will be to run the docker container. Since our application is structured with group collaboration in mind, containerization allows the app itself to download and install all dependencies to function. In the SourceCode Directory in the terminal, enter the command *docker compose up --build*

This will initiate the docker container to build the database, app server, and online website. Docker will move through the requirements.txt file automatically and build all resources and dependencies that the app needs. Once the app is built, the terminal will show messages that the database tables have been initialized and are prepared for platform use. Navigate in the browser to *localhost:6455* to view the app built locally.

```
bunny@SL1Mshady:~/projects/VulcanActivityTracker/SourceCode$ docker compose up --build
[+] Building 2.4s (5/12)
=> [internal] load local image definitions 0.0%
=> == reading from stdin 0.0%
=> [internal] load build definition from Dockerfile 0.0%
=> == transferring dockerfile: 833B 0.0%
=> [internal] load metadata for docker.io/library/oraclelinux:9 1.2%
=> [internal] load dockerignore 0.0%
=> == transferring context: 2B 0.0%
=> [1/7] FROM docker.io/library/oraclelinux:9@sha256:ddd334fcd31a1596d4b6e4a43efe4ccb800fb9b8de05c8a73d6cbae4f43 1.0%
=> == resolve docker.io/library/oraclelinux:9@sha256:ddd334fcd31a1596d4b6e4a43efe4ccb800fb9b8de05c8a73d6cbae4f43 0.0%
=> == sha256:1be8039164bd3e1892606858cb97e3c441e27fe95691905d05c9071741dde1dc 9.44MB / 92.44MB 0.9%
=> [internal] load build context 0.0%
=> == transferring context: 3.79MB 0.0%
```

Figure 21: Docker initializing build

```
vulcan_platform_dev | DB connection successful
vulcan_platform_dev | Starting schema initialization...
vulcan_platform_dev | Table 'user' already exists. Skipping creation.
vulcan_platform_dev | user table schema initialized or updated.
2026-03-31 23:56:11.16 spid25s | Service Broker manager has started.
vulcan_platform_dev | Table 'activity' already exists. Skipping creation.
vulcan_platform_dev | activity table schema initialized or updated.
vulcan_platform_dev | Table 'friend' already exists. Skipping creation.
vulcan_platform_dev | activity table schema initialized or updated.
2026-03-31T23:56:11.207Z [dotnet /app/ASDE.Dotnet.dll] [154] ASDE.Dotnet starting up
mssql_server | 2026-03-31T23:56:11.215Z [dotnet /app/ASDE.Dotnet.dll] [154] MSSQL_LCID:
vulcan_platform_dev | Table 'clubs' already exists. Skipping creation.
vulcan_platform_dev | clubs table schema initialized or updated.
2026-03-31T23:56:11.267Z [dotnet /app/ASDE.Dotnet.dll] [154] Creation of
```

Figure 22: Database tables built, signals app is ready to access

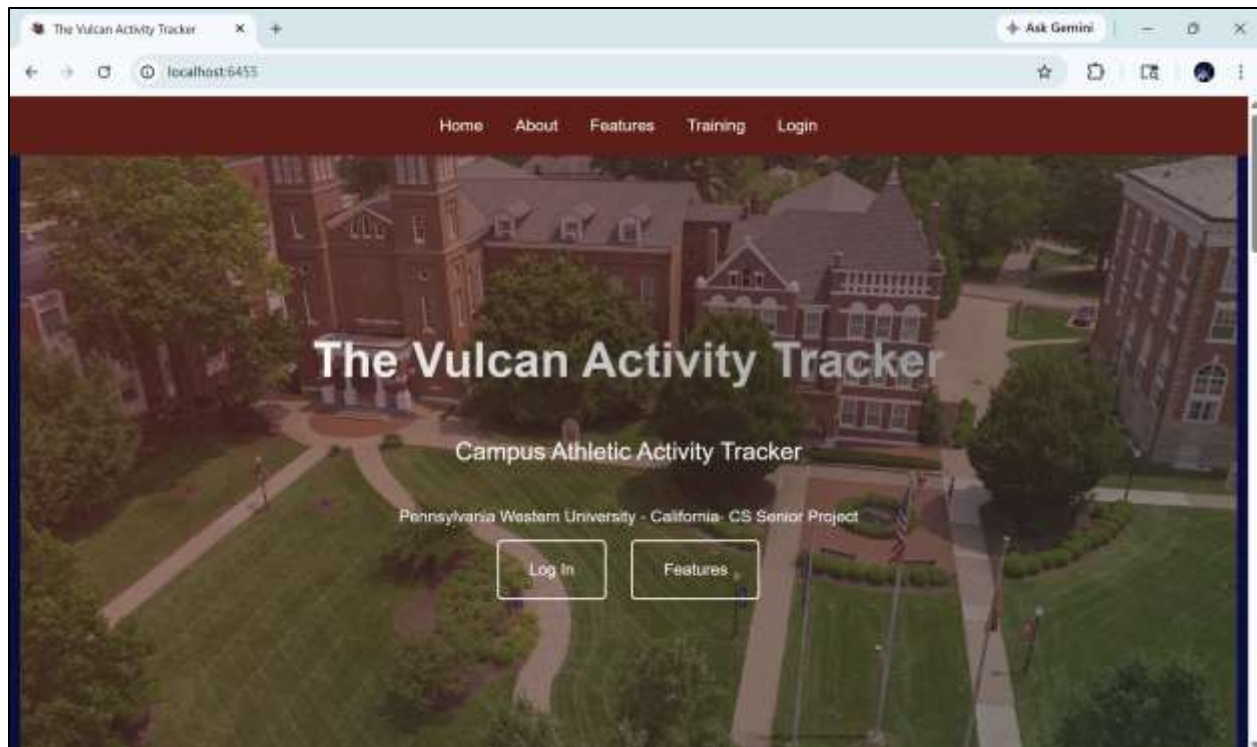


Figure 23: App is accessed by localhost:6455 in browser

```
Gracefully Stopping... press Ctrl+C again to force
Container vulcan_platform_dev Stopping
vulcan_platform_dev | [2026-03-31 23:57:25 +0000] [1] [INFO] Handling signal: term
vulcan_platform_dev | [2026-03-31 23:57:25 +0000] [7] [INFO] Worker exiting (pid: 7)
Container mssql_server Killing
Container vulcan_platform_dev Killing
^CContainer vulcan_platform_dev Stopped
Container mssql_server Stopping
Container vulcan_platform_dev Error Error while Killing
Container mssql_server Error Error while Killing
Container mssql_server Stopped
bunny@SLIMshady:~/projects/VulcanActivityTracker/SourceCode$ ^C
```

Figure 24: App is stopped locally by Ctrl-C key combination

This concludes the developer instructions portion of this document. Please reach out to the development team with any questions or errors regarding the local building process.

Team Contact Information:

Developer	Email
Margo Bonal	bon8330@pennwest.edu
John Gerega	ger9822@pennwest.edu
Luke Ruffing	RUF96565@pennwest.edu

List of References

Agafonkin, V. (2025, Aug 16th). Leaflet.

<https://leafletjs.com/>

Hevy Studios S.L. (2025). *Hevy – Gym Log Workout Tracker* (Version 2.4.7) [Mobile app].

<https://www.hevyapp.com/>

Lucidchart Software Inc. (2025). *Lucidchart* [Website and Software Application]

<https://www.lucidchart.com>

Nike, Inc. (2025). *Nike Run Club: Running Coach* (Version 7.70.0 for iOS) [Mobile app].

<https://www.nike.com/nrc-app>

OpenStreetMap. (2025). OpenStreetMap.

<https://www.openstreetmap.org/copyright>

The Run Buddy Ltd. (2022). *Runna: Running Training Plans* (Version 8.5.0) [Mobile app].

[https://www.runna.com/?gad_source=1&gad_campaignid=22530268929&gbraid=0AAA
AABfUKAUcs2aA7jayQhjZYrJnvDftf&gclid=CjwKCAjwuePGBhBZEiwAIGCVS1dta
MKRlGDzWrJsTvy5TKab0O_LodHzXuZie-qwF0uvQrYvfe_-CRoC89MQAvD_BwE](https://www.runna.com/?gad_source=1&gad_campaignid=22530268929&gbraid=0AAA
AABfUKAUcs2aA7jayQhjZYrJnvDftf&gclid=CjwKCAjwuePGBhBZEiwAIGCVS1dta
MKRlGDzWrJsTvy5TKab0O_LodHzXuZie-qwF0uvQrYvfe_-CRoC89MQAvD_BwE)

STRONG FITNESS PTE. LTD. (2025). *Strong Workout Tracker Gym Log* (Version 6.2.1)
[Mobile app].

<https://www.strong.app/>

Strava, Inc. (2025, Sept 11). *Strava (Version 428.0.1)* [Website and Mobile App].

<https://www.strava.com>

Table of Index / Glossary

- **Vulcan Activity Tracker** – A fitness and social activity tracking application designed for PennWest students to log, monitor, and share athletic activities.
- **User Interface (UI)** – The front-end portion of the application that users interact with through a web browser.
- **Backend Server (Flask)** – The server-side component responsible for handling API requests, processing data, and managing application logic.
- **Database (MSSQL Server)** – The system used to store and retrieve user data, activities, clubs, and other application information.
- **Authentication Module** – The system responsible for verifying user credentials during login.
- **Activity Management Module** – Handles creation, retrieval, updating, and deletion of user activity data.
- **Group Management Module** – Manages clubs, teams, memberships, and user roles.
- **Leaderboard Module** – Processes and displays rankings of users based on activity metrics.
- **Database Module** – Responsible for storing and retrieving all application data.
- **Dashboard** – The main user page displaying activity summaries, clubs, friends, and challenges.
- **Training Page** – A feature where users log and view fitness activities with sport-specific metrics.
- **Maps Page** – Allows users to create, view, and manage routes, as well as view weather forecasts.

- Clubs Page – Enables users to create, join, and interact with recreational or organizational groups.
- Teams Page – A restricted feature for official team members and coaches to manage team-related content.
- Leaderboard – A ranking system that compares users based on activity performance metrics.
- Challenges – User-created or public competitions that track progress toward fitness goals.
- Settings Page – Allows users to manage account details such as username, email, and account status.
- Public Activity – Activity data visible to other users on the platform.
- Private Activity – Activity data restricted from other users' view.
- API (Application Programming Interface) – A set of endpoints used for communication between the frontend and backend.
- HTTP Requests – Messages sent between client and server to request or send data.
- Data Propagation – The movement of data from the frontend to the backend and into the database.
- Agile Development – A software development methodology emphasizing iterative progress and flexibility.
- Waterfall Model – A linear development approach used for documentation and planning stages.
- Gantt Chart – A project management tool used to visualize timelines and task progress.

- Route Builder – A feature that allows users to draw and save custom activity routes on a map.
- Weather API – External service providing real-time weather data for activity planning.
- Map API (Leaflet) – External tool used to render maps and support route creation.
- Account Creation – The process of registering a new user using email, username, and password.
- Login Credentials – Username and password used to access the application.
- Encryption – Security method used to protect user data, especially login information.
- Session – A temporary state that keeps a user logged into the system.
- Activity History – A record of past activities logged by a user.
- Search Filter – A tool used to find activities based on keywords like date, calories, or notes.
- Friend Feature – Allows users to follow others and view their public activities.
- Club Details Page – Displays information about a specific club including members and activities.
- Team Invite – A request sent by a coach to allow a user to join a team.
- Mini Leaderboard – A smaller ranking system within clubs or teams.
- Progress Bar – Visual representation of a user's advancement in a challenge.
- Case-Sensitive Search – A search function that distinguishes between uppercase and lowercase letters.
- Frontend – The client-side of the application responsible for display and interaction.
- Backend – The server-side of the application responsible for logic and data handling.

- Data Validation – The process of ensuring input data is correct and properly formatted before storage.
- Open-Source Tools – Software resources freely available for development use.
- Real-Time Processing – Immediate handling and updating of data without delays.
- Asynchronous Functions – JavaScript functions that run independently of the main program flow.
- Environment Variables – Configuration values used by the application at runtime.

Appendix I: Writing Center report

Writing Center: California Campus

Appointment: Wed, 4/8/2026

Consultant Name: Tatiana Kostovny

Report:

The structure of the manual was easy to follow and left no confusion for the reader. It was easy to understand why the project was created, how it was created, and how it is intended to be used. The only issues within the manual were simple grammar and spelling mistakes throughout. I would go through it one more time to make sure that those are all corrected before turning in the document.

Appendix II: Team Details

This User Manual document was developed through the collaboration of all team members. The document was made with the combined efforts of the entire team over the course of the project timeline. While each member contributed across various parts of the document, the primary contributors for each section are listed below:

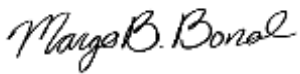
- Margo Bonal:
 - Title Page, table of contents, team details, authentication, developer instructions, user instructions, project overview, screenshots, SWE Principles, comparison, motivation,
- John Gerega:
 - Teams page, Glossary
- Luke Ruffing:
 - Clubs page

The rest of the major sections of the document were completed collaboratively with all members participating equally in discussions, decision making, and writing. This collaborative approach taken to completing the document allowed the group to refine ideas and reach a consensus before making any big decisions.

In addition to the writing of each section, all team members took part in proof reading and formatting ensuring the creation of a clear, concise and well-organized document. Together the team worked to check for proper grammar, proper citation, and adherence to all formatting guidelines.

Appendix III: Workflow Authentication

I, Margo Bonal, confirm that all work I have contributed to this document has been to the best of my knowledge, completed with integrity and in accordance with the objectives and collaborative standards established by the team. I attest that I will maintain these standards in all future contributions to this project.

Signature: 

Date: 4/26/26

I, John Gerega confirm that all work I have contributed to this document has been to the best of my knowledge completed with integrity and in accordance with the objectives and collaborative standards established by the team. I attest that I will maintain these standards in all future contributions to this project.

Signature: 

Date: 4/26/26

I, Luke Ruffing, confirm that all work I have contributed to this document has been to the best of my knowledge completed with integrity and in accordance with the objectives and collaborative standards established by the team. I attest that I will maintain these standards in all future contributions to this project.

Signature: 

Date: 4/26/26

Appendix III: Code listing

This section is a summary of the files used in the Vulcan Activity Trackers source code.

This section lists each file in **bold** and gives the number of classes and functions in each file when applicable, and a short description of its use. For source files function headers are included and put in **bold**. For support files a short file description is provided.

File: .env

Contains 0 classes and 0 functions.

This file stores the environment variables used by the application, such as database credentials and API keys. It supports local configuration and deployment.

File: .gitignore

Contains 0 classes and 0 functions.

This file lists files and folders that Git should ignore. It helps prevent temporary or sensitive files from being tracked.

File: app.py

Contains 0 classes and 0 functions.

This file is the main entry point of the Flask application. It creates the app registers blueprints, and initializes the database schema at startup.

File: cleanup.sh

Contains 0 classes and 0 functions.

This file is a maintenance script used to scale the Azure Container App.

File: deploy.sh

Contains 0 classes and 0 functions.

This file is a deployment script used to create Azure resources, build and push the Docker image, and deploy the application. It mainly uses az, docker, and environment values from .env.

File: docker-compose.yml

Contains 0 classes and 0 functions.

This file defines the multi-container setup for the application. It describes how the services should run together.

File: Dockerfile

Contains 0 classes and 0 functions.

This file defines how the application container image is built. It supports packaging the Flask application for deployment.

File: requirements.txt

Contains 0 classes and 0 functions.

This file lists the Python packages required by the project. It supports dependency installation and environment setup.

File: .devcontainer/devcontainer.json

Contains 0 classes and 0 functions.

This file defines the development container configuration for the project. It supports a consistent coding environment.

File: server/api/activity_api.py

Contains 0 classes and 8 functions:

def enter_activity()

This function accepts activity data submitted by a user and stores it in the database. It mainly uses `request.get_json()`, `get_db_connection()`, `get_user_id()`, and `insert_activity()`.

def fill_activity()

This function returns the current users activity history for the training page. It mainly uses `request.get_json()`, `get_db_connection()`, `get_user_id()`, and `get_user_activities()`.

def fill_Dashactivity()

This function returns activity data for the dashboard activity feed. It mainly uses `request.get_json()`, `get_db_connection()`, `get_user_id()`, and `get_user_activities()`.

def fill_FriendActivity()

This function returns public activity data for another user or friend view. It mainly uses `request.get_json()`, `get_db_connection()`, `get_user_id()`, and `get_public_activities()`.

def public_leaderboard()

This function returns leaderboard data, optionally filtered by sport. It mainly uses `request.args.get()`, `get_db_connection()`, and `get_public_leaderboard()`.

def get_public_leaderboard(conn, sport_type=None)

This function builds the public leaderboard using activity records from the database. It mainly works with SQL cursor operations and result formatting.

def get_specific_sport_data(conn, sport)

This function is used to isolate sport-specific leaderboard logic. It supports the public leaderboard workflow.

File: server/api/challenges_api.py

Contains 0 classes and 7 functions:

def create_challenge()

This function creates a new challenge after validating the submitted information. It mainly uses `request.get_json()`, `get_db_connection()`, `get_user_id()`, `challenge_name_exists()`, and `insert_challenge()`.

def list_challenges()

This function returns the challenges visible to the current user that they have not joined. It mainly uses `finalize_expired_challenges()`, `get_visible_not_user_challenges()`, and `get_user_id()`.

def my_challenges()

This function returns the challenges created by or joined by the current user. It mainly uses `finalize_expired_challenges()`, `get_visible_user_challenges()`, `get_user_medals()`, and `get_user_id()`.

def join_challenge()

This function adds the current user to a selected challenge. It mainly uses `add_participant_to_challenge()` and `get_user_id()`.

def leave_challenge()

This function removes the current user from a selected challenge. It mainly uses `remove_participant_from_challenge()` and `get_user_id()`.

def delete_challenge()

This function deletes a challenge, usually when requested by the owner. It mainly uses `remove_challenge_from_database()` and `get_user_id()`.

def challenge_detail()

This function returns detailed information about one challenge, including participants and leaderboard data. It mainly uses `get_all_challenges()`, `get_participant_details()`, and `get_challenge_leaderboard()`.

File: server/api/club_api.py

Contains 0 classes and 12 functions:

def create_club()

This function creates a new club from user-submitted form data. It mainly uses `insert_club()` and `get_user_id()`.

def list_clubs()

This function lists clubs that the current user has not joined. It mainly uses `get_not_user_clubs()`, `get_user_pending_club_request()`, and `get_user_id()`.

def my_clubs()

This function lists clubs that the current user belongs to. It mainly uses `get_user_clubs()`, `get_user_pending_club_request()`, and `get_user_id()`.

def join_club()

This function allows a user to join a public club. It mainly uses `get_all_clubs()`, `add_member_to_club()`, and `get_user_id()`.

def leave_club()

This function removes a user from a club membership list. It mainly uses `remove_member_from_club()` and `get_user_id()`.

def delete_club()

This function deletes a club when requested by its owner. It mainly uses `remove_club_from_database()` and `get_user_id()`.

def club_detail()

This function returns detailed club information, including members, leaderboard data, and recent activity. It mainly uses `get_all_clubs()`, `get_club_this_week_leaderboard()`, `get_club_last_week_leaders()`, `get_club_recent_activities()`, `usernames_from_userids()`, and `get_user_pending_club_request()`.

def request_join_club()

This function creates a pending join request for a private club. It mainly uses `create_club_join_request()`, `get_all_clubs()`, and `get_user_pending_club_request()`.

def cancel_join_request()

This function removes a previously submitted join request. It mainly uses `cancel_club_join_request()`.

def club_requests_for_owner()

This function returns pending club join requests for the current club owner. It mainly uses `get_pending_club_requests_for_owner()`.

def accept_club_request_route()

This function accepts a pending club join request. It mainly uses `accept_club_join_request()`.

def decline_club_request_route()

This function declines a pending club join request. It mainly uses `decline_club_join_request()`.

File: server/api/dash_api.py

Contains 0 classes and 8 functions:

def sendFriendsList()

This function returns the friend list and related user data for the dashboard. It mainly uses `fetch_all_users()`, `get_users_friends()`, and `get_user_id()`.

def add_friend()

This function stores a new friendship between the current user and another user. It mainly uses `insert_friend()` and `get_user_id()`.

def like_friend()

This function toggles a like for another user or friend card. It mainly uses `check_if_liked()`, `toggle_like_friend()`, `get_total_likes()`, and `get_users_friends()`.

def get_likes_count()

This function returns the total number of likes for a selected friend. It mainly uses `get_total_likes()`.

def thumbs_up_friend()

This function toggles a thumbs-up reaction on an activity. It mainly uses `check_if_thumbs_up()`, `toggle_thumbs_up()`, and `get_total_thumbs_up()`.

def get_thumb_count()

This function returns the number of thumbs-up reactions on an activity. It mainly uses `get_total_thumbs_up()`.

def fill_dash_clubs()

This function returns club data for the dashboard overview. It mainly uses `get_user_clubs()` and `get_user_id()`.

def fill_dash_challenges()

This function returns challenge data for the dashboard overview. It mainly uses `get_dash_challenges()` and `get_user_id()`.

File: server/api/login_api.py

Contains 0 classes and 3 functions:

def login()

This function accepts the initial login request and prepares account lookup data. It mainly uses `fetch_all_users()`, `fetch_user_by_username()`, `store_login()`, and `link_coach_to_team()`.

def verify()

This function verifies the password and sets up related role or team data. It mainly uses `fetch_user_by_username()`, `assign_coach_role_if_match()`, and `link_coach_to_team()`.

def logout()

This function clears the current session and signs the user out. It mainly uses `session.pop()` and `jsonify()`.

File: server/api/maps_api.py

Contains 0 classes and 4 functions:

def storeMapRoutes()

This function stores a named route created by the user on the map page. It mainly uses `add_route()` and `get_user_id()`.

def getUserRoutes()

This function returns all saved routes for the current user. It mainly uses `get_user_routes()` and `get_user_id()`.

def deleteRoute()

This function removes one saved route from storage. It mainly uses `delete_route()` and `get_user_id()`.

def sendweatherkey()

This function returns the weather API key used by the frontend. It mainly uses `os.getenv()` and `jsonify()`.

File: server/api/password_api.py

Contains 0 classes and 2 functions:

def reset_password(token)

This function handles the reset-password form after the user follows a reset link. It mainly uses `verify_reset_token()`, `render_template()`, `url_for()`, and password hashing.

def request_password_reset()

This function creates a reset token and sends a password-reset email. It mainly uses `generate_reset_token()`, `send_reset_email()`, and `url_for()`.

File: server/api/protectRoutes.py

Contains 0 classes and 1 function:

def login_required(view_func)

This function protects a page route by redirecting users who are not logged in. It mainly uses `redirect()`, `url_for()`, and the wrapped view function.

File: server/api/register_api.py

Contains 0 classes and 1 function:

def register()

This function begins the registration process and sends email verification. It mainly uses `generate_email_verification_token()` and `send_verification_email()`.

File: `server/api/settings_api.py`

Contains 0 classes and 4 functions:

`def change_username()`

This function updates the current user's username in the database and session. It mainly uses SQL cursor operations, session data, and `jsonify()`.

`def delete_account()`

This function deletes a user account and removes related records such as clubs, activities, likes, and session data. It mainly uses direct SQL operations, `session.clear()`, and transaction control.

`def activity_stat()`

This function returns quick settings statistics such as total activities, challenges, clubs, and email. It mainly uses `get_user_id()`, `get_user_activities()`, `get_user_challenges()`, `get_user_clubs()`, and `get_user_email()`.

`def get_user_info()`

This function returns the current user's display name and email address. It mainly uses SQL cursor operations and session data.

File: server/api/team_api.py

Contains 0 classes and 13 functions:

def create_team_route()

This function creates a team for the logged-in user. It mainly uses create_team_store() and session data.

def invite_to_team()

This function sends a team invitation to another user. It mainly uses get_user_id(), invite_user_to_team(), and coach membership checks.

def my_teams()

This function returns the teams for the current user. It mainly uses get_user_teams().

def my_invites()

This function returns pending team invitations for the current user. It mainly uses get_pending_team_invites().

def accept_invite_route()

This function accepts a pending team invitation. It mainly uses `accept_team_invite()`.

def decline_invite_route()

This function declines a pending team invitation. It mainly uses `decline_team_invite()`.

def team_detail()

This function returns detailed team data, including roster and access checks. It mainly uses `user_can_access_team()` and direct roster queries.

def list_all_teams()

This function returns all teams for the teams page. It mainly uses SQL cursor operations and `jsonify()`.

def get_team(team_id)

This function returns one team's data after verifying access permissions. It mainly uses `user_can_access_team()` and `get_team_details()`.

def create_announcement_route()

This function creates a team announcement. It mainly uses `is_user_team_coach()` and `create_team_announcement()`.

def get_announcements_route(team_id)

This function returns announcements for a team. It mainly uses `user_can_access_team()` and `get_team_announcements()`.

def create_schedule_event_route()

This function creates a new event in the team schedule. It mainly uses `is_user_team_coach()` and `create_team_schedule_event()`.

def get_schedule_route(team_id)

This function returns schedule events for a team. It mainly uses `user_can_access_team()` and `get_team_schedule()`.

def team_leaderboard(team_id)

This function returns team leaderboard data with optional sport filtering. It mainly uses `user_can_access_team()` and `get_team_leaderboard()`.

File: server/api/verify_api.py

Contains 0 classes and 1 function:

def verify_email(token)

This function checks whether an email-verification token is valid. It mainly uses `confirm_email_verification_token()`.

File: server/blueprints/blue.py

Contains 0 classes and 6 functions:

def make_view(tpl, active_page)

This function creates reusable page-rendering views from the route table. It mainly uses `render_template()` and `login_required()`.

def club_detail(club_name)

This function renders the club detail page. It mainly uses `render_template()`.

def create_club()

This function renders the create-club page. It mainly uses `render_template()`.

def challenge_detail(challenge_name)

This function renders the challenge detail page. It mainly uses `render_template()`.

def team_detail(team_id)

This function renders the team detail page. It mainly uses `render_template()`.

def create_challenge()

This function renders the create-challenge page. It mainly uses `render_template()`.

File: server/controllers/activity_store.py

Contains 0 classes and 3 functions;

def insert_activity(conn, user_id: str, data: dict)

This function inserts a new activity into the database. It mainly uses SQL insert operations.

def get_user_activities(conn, user_id: str)

This function returns all activities for one user. It mainly uses SQL queries and result formatting.

def get_public_activities(conn, user_id)

This function returns public activity feed data. It mainly uses SQL queries and response formatting.

File: server/controllers/challenges_store.py

Contains 0 classes and 18 functions:

def insert_challenge(conn, user_id, name, description, ...)

This function inserts a new challenge into the database. It mainly uses SQL insert logic.

def get_all_challenges(conn)

This function returns all challenge records. It supports detail lookups and challenge browsing.

def get_not_user_challenges(conn, user_id)

This function returns challenges the current user has not joined. It supports the challenge browse view.

def get_user_challenges(conn, user_id)

This function returns challenges related to the current user. It supports the user's personal challenge view.

def get_dash_challenges(conn, user_id)

This function returns challenge data for the dashboard. It supports the dashboard challenge section.

def add_participant_to_challenge(conn, challenge_id, user_id)

This function adds a user to a challenge. It supports the join challenge action.

def remove_participant_from_challenge(conn, challenge_id, user_id)

This function removes a user from a challenge. It supports the leave challenge action.

def remove_challenge_from_database(conn, challenge_id, user_id)

This function deletes a challenge from the database. It supports the owner delete workflow.

def calculate_challenge_progress(conn, challenge, user_id)

This function calculates a participant's progress toward a challenge goal. It supports leaderboard and detail views.

def challenge_name_exists(conn, challenge_name)

This function checks whether a challenge name already exists. It supports challenge validation.

def get_participant_details(conn, user_ids)

This function expands participant IDs into usernames and related detail records. It supports the challenge detail page.

def get_challenge_leaderboard(conn, challenge)

This function builds leaderboard data for a challenge. It supports ranking display.

def get_user_medals(conn, user_id)

This function returns medals earned by a user. It supports the my challenges view.

def add_medal_to_user(conn, user_id, medal_type)

This function stores a medal award for a user. It supports challenge completion logic.

def award_challenge_medals(conn, challenge)

This function determines winners and assigns medals for a completed challenge. It mainly uses `add_medal_to_user()`.

def finalize_expired_challenges(conn)

This function closes expired challenges and runs completion logic. It mainly uses `award_challenge_medals()`.

`def get_visible_not_user_challenges(conn, user_id)`

This function filters visible challenges that the current user is not part of. It supports the challenge browse page.

`def get_visible_user_challenges(conn, user_id)`

This function filters visible challenges belonging to the current user. It supports the my-challenges page.

File: `server/controllers/club_store.py`

Contains 0 classes and 23 functions:

`def insert_club(conn, user_id, name, description, sport_type, privacy)`

This function inserts a new club into the database. It supports the club creation workflow.

`def get_all_clubs(conn)`

This function returns all club records from the database. It supports club lookups and detail views.

def get_not_user_clubs(conn, user_id)

This function returns clubs that the current user has not joined. It supports the public club browsing page.

def get_user_clubs(conn, user_id)

This function returns the clubs that belong to or include the current user. It supports the my clubs and dashboard views.

def add_member_to_club(conn, club_id, user_id, username)

This function adds a user to a club membership list. It supports the join club workflow.

def remove_member_from_club(conn, club_id, user_id, username)

This function removes a user from a club membership list. It supports the leave club workflow.

def remove_club_from_database(conn, club_id, user_id)

This function deletes a club from the database. It supports the owner delete action.

def usernames_from_userids(conn, user_ids)

This function converts stored user IDs into usernames. It supports member display in club detail views.

def club_uses_distance_ranking(sport_type)

This function decides whether a club leaderboard should use distance-based scoring. It supports leaderboard formatting and ranking rules.

def get_club_member_details(conn, club)

This function builds detailed member information for a club. It supports the club detail page.

def get_week_range(reference_date=None)

This function calculates the date boundaries for a weekly leaderboard period. It supports current-week and last-week club rankings.

def calculate_club_member_stats(conn, user_id, sport_type, start_date, end_date)

This function calculates one club member's stats for a selected date range. It supports weekly leaderboard generation.

def get_club_leaderboard_for_range(conn, club, start_date, end_date)

This function builds leaderboard data for a club across a specified time range. It mainly uses `calculate_club_member_stats()`.

def get_club_this_week_leaderboard(conn, club)

This function returns the current week's club leaderboard. It mainly uses `get_club_leaderboard_for_range()`.

def get_club_last_week_leaders(conn, club)

This function returns the previous week's top club results. It mainly uses `get_club_leaderboard_for_range()`.

def create_club_join_request(conn, club_id, requesting_user_id, club_owner_user_id)

This function stores a pending join request for a private club. It supports the private club request workflow.

def cancel_club_join_request(conn, club_id, requesting_user_id)

This function removes a pending club join request. It supports the cancel request action.

def get_pending_club_requests_for_owner(conn, owner_user_id)

This function returns pending requests for clubs owned by a user. It supports the inbox owner view.

def get_user_pending_club_request(conn, club_id, requesting_user_id)

This function checks whether a user already has a pending request for a club. It supports button state logic in the club interface.

def accept_club_join_request(conn, club_id, requesting_user_id, owner_user_id)

This function accepts a club join request and updates membership data. It supports the owner approval workflow.

def decline_club_join_request(conn, club_id, requesting_user_id, owner_user_id)

This function declines a club join request. It supports the owner moderation workflow.

def get_username_map_for_user_ids(conn, user_ids)

This function builds a mapping between user IDs and usernames. It supports formatted display of club member information.

```
def get_club_recent_activities(conn, club, limit=25)
```

This function returns recent activities related to a club. It supports the recent activity section on the club detail page.

File: server/controllers/email_store.py

Contains 0 classes and 1 function:

```
def send_reset_email(user_email, reset_link)
```

This function sends a password reset email to the user. It supports the password recovery workflow.

File: server/controllers/friend_store.py

Contains 0 classes and 2 functions:

def insert_friend(conn, current_user: str, friend_user: str) -> None

This function creates a friendship record between two users. It supports the add friend feature.

def get_users_friends(conn, user_id: str) -> list

This function returns the friend list for a user. It supports the dashboard and social features.

File: server/controllers/like_store.py

Contains 0 classes and 6 functions:

def toggle_like_friend(conn, user_id, friend_id)

This function toggles a like for another user. It supports the friend like feature.

def check_if_liked(conn, user_id, friend_id)

This function checks whether a user has already liked another user. It supports like button state logic.

```
def get_total_likes(conn, friend_id)
```

This function returns the total number of likes for a user. It supports the dashboard like counter.

```
def check_if_thumbs_up(conn, liker_id: str, activity_id: str) -> bool
```

This function checks whether a user has already given a thumbs-up to an activity. It supports activity reaction logic.

```
def get_total_thumbs_up(conn, activity_id: str) -> int
```

This function returns the total thumbs-up count for an activity. It supports activity feed counters.

```
def toggle_thumbs_up(conn, liker_id: str, activity_id: str)
```

This function toggles a thumbs-up reaction on an activity. It supports the activity social interaction feature.

File: server/controllers/login_store.py

Contains 0 classes and 3 functions:

def store_login(conn, email, username, passwordHash)

This function stores user login information in the database. It supports account creation and login setup.

def fetch_login(conn, username)

This function returns login information for one user. It supports credential lookup.

def fetch_all_users(conn)

This function returns all users from the database. It supports login checks and social features.

File: server/controllers/map_store.py

Contains 0 classes and 3 functions:

def add_route(conn, user_id, route_name, distance, coordinates)

This function stores a saved route for a user. It supports the map route save feature.

def get_user_routes(conn, user_id)

This function returns all saved routes for a user. It supports loading routes on the maps page.

def delete_route(conn, user_id, RouteName)

This function deletes one saved route for a user. It supports the route removal workflow.

File: server/controllers/team_store.py

Contains 0 classes and 7 functions:

def create_team(conn, coach_user_id, team_name, sport, description)

This function creates a new team in the database. It supports the team creation workflow.

def invite_user_to_team(conn, team_id, coach_user_id, invited_user_id)

This function stores a team invitation for another user. It supports the invite player workflow.

def accept_team_invite(conn, team_id, user_id)

This function accepts a pending team invitation. It supports the inbox accept action.

def decline_team_invite(conn, team_id, user_id)

This function declines a pending team invitation. It supports the inbox decline action.

def get_user_teams(conn, user_id)

This function returns teams that belong to the user. It supports the my teams feature.

def get_pending_team_invites(conn, user_id)

This function returns pending team invites for the user. It supports the inbox view.

def get_team_detail_for_user(conn, team_id, user_id)

This function returns detailed team information for an authorized user. It supports secure team detail access.

File: server/controllers/user_store.py

Contains 0 classes and 2 functions:

def get_user_id(conn, username)

This function returns the database user ID for a username. It supports many backend workflows that need user lookup.

def get_user_email(conn, user_id)

This function returns the email address for a user ID. It supports settings and account features.

File: server/database/connect.py

Contains 0 classes and 1 function:

def get_db_connection()

This function opens the database connection and retries if the service is not ready yet. It is the main shared database access point for the backend.

File: server/database/team_queries.py

Contains 0 classes and 20 functions:

def assign_coach_role_if_match(conn, user_id, email)

This function checks whether a user should be assigned a coach role based on email data. It supports team role setup during login.

def is_user_on_team(conn, user_id, team_id)

This function checks whether a user belongs to a team. It supports access control logic.

```
def is_user_team_coach(conn, user_id, team_id)
```

This function checks whether a user has coach or admin privileges on a team. It supports restricted team actions.

```
def user_can_view_team(conn, team_id, user_id, email=None)
```

This function determines whether a user is allowed to view a team. It supports team privacy and access control.

```
def get_user_teams(conn, user_id)
```

This function returns team records for a user. It supports team listing features.

```
def get_team_members(conn, team_id)
```

This function returns the members of a team. It supports team detail and roster displays.

```
def fetch_user_by_username(conn, username)
```

This function returns one user record by username. It supports login and user lookup features.

```
def get_user_id(conn, username)
```

This function returns the database user ID for a username. It supports several team workflows.

def create_team_invite(conn, team_id, invited_user_id, invited_by_user_id)

This function stores a team invite record. It supports the invitation workflow.

def get_pending_team_invites(conn, user_id)

This function returns pending team invites for a user. It supports the inbox page.

def accept_team_invite(conn, invite_id, user_id)

This function accepts a stored team invite. It supports invitation approval.

def decline_team_invite(conn, invite_id, user_id)

This function declines a stored team invite. It supports invitation rejection.

def user_can_access_team(conn, team_id, user_id)

This function performs the main access check before team data is shown. It supports secure team page access.

def get_team_details(conn, team_id)

This function returns the full detail object for a team. It supports the team detail route.

def link_coach_to_team(conn, user_id, email)

This function links a coach account to matching team data. It supports team role setup.

def create_team_announcement(conn, team_id, created_by_user_id, title, body)

This function inserts a new team announcement. It supports the coach announcement feature.

def get_team_announcements(conn, team_id)

This function returns announcements for a team. It supports the team detail page.

**def create_team_schedule_event(conn, team_id, created_by_user_id, event_title,
event_type, event_date, location, notes)**

This function stores a new team schedule event. It supports practice, game, and meeting scheduling.

def get_team_schedule(conn, team_id)

This function returns the schedule for a team. It supports the team detail page.

def get_team_leaderboard(conn, team_id, sport_type='all')

This function returns leaderboard data for a team. It supports team performance views and sport filtering.

File: server/database/tokens.py

Contains 0 classes and 3 functions:

def _serializer()

This function creates the serializer used for secure token generation and validation. It supports password reset token handling.

def generate_reset_token(email)

This function creates a password reset token for a user email. It supports password recovery.

def verify_reset_token(token, expiration=3600)

This function validates a password reset token and returns its email data if valid. It supports the reset-password workflow.

File: server/database/__init__.py

Contains 0 classes and 1 function:

def init_or_upgrade_schema(conn)

This function creates or upgrades all required database tables. It supports startup initialization of the full database.

File: server/database/schema/activityLikeSchema.py

Contains 0 classes and 1 function:

def create_activity_likes_table(conn)

This function creates the activity likes table if it does not already exist. It supports initial database setup.

File: server/database/schema/activitySchema.py

Contains 0 classes and 1 function:

def create_activity_table(conn)

This function creates the activity table if it does not already exist. It supports initial database setup.

File: server/database/schema/announcementSchema.py

Contains 0 classes and 1 function:

def create_team_announcements_table(conn)

This function creates the team announcements table if it does not already exist. It supports initial database setup.

File: server/database/schema/challengesSchema.py

Contains 0 classes and 1 function:

def create_challenges_table(conn)

This function creates the challenges table if it does not already exist. It supports initial database setup.

File: server/database/schema/clubInviteSchema.py

Contains 0 classes and 1 function:

def create_clubs_invites_table(conn)

This function creates the club requests table if it does not already exist. It supports initial database setup.

File: server/database/schema/clubSchema.py

Contains 0 classes and 1 function:

def create_clubs_table(conn)

This function creates the clubs table if it does not already exist. It supports initial database setup.

File: server/database/schema/friendSchema.py

Contains 0 classes and 1 function:

def create_friends_table(conn)

This function creates the friends table if it does not already exist. It supports initial database setup.

File: server/database/schema/likeSchema.py

Contains 0 classes and 1 function:

def create_likes_table(conn)

This function creates the likes table if it does not already exist. It supports initial database setup.

File: server/database/schema/loginSchema.py

Contains 0 classes and 1 function:

def create_user_table(conn)

This function creates the user table if it does not already exist. It supports initial database setup.

File: server/database/schema/mapSchema.py

Contains 0 classes and 1 function:

def create_maps_table(conn)

This function creates the maps table if it does not already exist. It supports initial database setup.

File: server/database/schema/populateTeamSchema.py

Contains 0 classes and 1 function:

def seed_teams_and_coaches(conn)

This function inserts starter team and coach data into the database. It supports initial project setup and testing.

File: server/database/schema/scheduleSchema.py

Contains 0 classes and 1 function:

def create_team_schedule_table(conn)

This function creates the team schedule table if it does not already exist. It supports initial database setup.

File: server/database/schema/teamInviteSchema.py

Contains 0 classes and 1 function:

def create_team_invites_table(conn)

This function creates the team invites table if it does not already exist. It supports initial database setup.

File: server/database/schema/teamSchema.py

Contains 0 classes and 4 functions:

def create_teams_table(conn)

This function creates the teams table if it does not already exist. It supports initial database setup.

def create_team_member_table(conn)

This function creates the team members table if it does not already exist. It supports initial database setup.

def create_team_coach_emails_table(conn)

This function creates the coach emails table if it does not already exist. It supports initial database setup.

def create_team_join_requests_table(conn)

This function creates the team join requests table if it does not already exist. It supports initial database setup.

File: templates/base_temp/dashboard.html

Contains 0 classes and 0 functions:

This file is the main dashboard layout template. It provides the shared structure for logged-in dashboard pages.

File: templates/base_temp/welcome.html

Contains 0 classes and 0 functions:

This file is the public welcome page template. It serves as the main landing page for visitors.

File: templates/pages/createacc.html

Contains 0 classes and 0 functions:

This file is the account creation page template. It provides the registration form interface.

File: templates/pages/login.html

Contains 0 classes and 0 functions.

This file is the login page template. It provides the user authentication form.

File: templates/pages/reset_password.html

Contains 0 classes and 0 functions.

This file is the reset password page template. It provides the interface for entering a new password.

File: templates/pages/dash/challenges.html

Contains 0 classes and 0 functions.

This file is the main challenges dashboard page. It displays available and joined challenges.

File: templates/pages/dash/challenge_create.html

Contains 0 classes and 0 functions.

This file is the create challenge page template. It provides the challenge creation form.

File: templates/pages/dash/challenge_details.html

Contains 0 classes and 0 functions.

This file is the challenge detail page template. It displays one challenge's information and leaderboard.

File: templates/pages/dash/clubs.html

Contains 0 classes and 0 functions.

This file is the clubs dashboard page. It displays available and joined clubs.

File: templates/pages/dash/club_create.html

Contains 0 classes and 0 functions.

This file is the create club page template. It provides the club creation form.

File: templates/pages/dash/club_details.html

Contains 0 classes and 0 functions.

This file is the club detail page template. It displays members, rankings, and recent activity.

File: templates/pages/dash/home.html

Contains 0 classes and 0 functions.

This file is the dashboard home page template. It displays summary content for the logged-in user.

File: templates/pages/dash/inbox.html

Contains 0 classes and 0 functions.

This file is the inbox page template. It displays invitations and pending requests.

File: templates/pages/dash/leaderboard.html

Contains 0 classes and 0 functions.

This file is the public leaderboard page template. It displays ranked activity data.

File: templates/pages/dash/maps.html

Contains 0 classes and 0 functions.

This file is the maps page template. It displays saved routes and map tools.

File: templates/pages/dash/settings.html

Contains 0 classes and 0 functions.

This file is the settings page template. It displays account and statistics information.

File: templates/pages/dash/teams.html

Contains 0 classes and 0 functions.

This file is the teams page template. It displays available teams and browsing options.

File: templates/pages/dash/team_detail.html

Contains 0 classes and 0 functions.

This file is the team detail page template. It displays roster, schedule, leaderboard, and announcements.

File: templates/pages/dash/training.html

Contains 0 classes and 0 functions.

This file is the training page template. It provides the interface for entering and viewing activity data.

File: templates/style/css/activity.css

Contains 0 classes and 0 functions.

This file defines styles for the activity and training related views. It controls layout, spacing, and visual formatting.

File: templates/style/css/challenges.css

Contains 0 classes and 0 functions.

This file defines styles for the challenges pages. It controls challenge card and detail page appearance.

File: templates/style/css/clubs.css

Contains 0 classes and 0 functions.

This file defines styles for the clubs pages. It controls club listing and detail page appearance.

File: templates/style/css/dashboardStyle.css

Contains 0 classes and 0 functions.

This file defines shared styles for the dashboard layout. It controls the main logged-in interface appearance.

File: templates/style/css/homeStyle.css

Contains 0 classes and 0 functions.

This file defines styles for the home or welcome page. It controls the landing page presentation.

File: templates/style/css/inbox.css

Contains 0 classes and 0 functions.

This file defines styles for the inbox page. It controls invitation and request display formatting.

File: templates/style/css/leaderboard.css

Contains 0 classes and 0 functions.

This file defines styles for the leaderboard page. It controls table layout and ranking presentation.

File: templates/style/css/loginstyle.css

Contains 0 classes and 0 functions.

This file defines styles for the login and account pages. It controls authentication page appearance.

File: templates/style/css/map.css

Contains 0 classes and 0 functions.

This file defines styles for the maps page. It controls map layout, route display, and related interface elements.

File: templates/style/css/modal.css

Contains 0 classes and 0 functions.

This file defines styles for modal windows. It controls popup positioning and visual behavior.

File: templates/style/css/settings.css

Contains 0 classes and 0 functions.

This file defines styles for the settings page. It controls profile and quick statistics presentation.

File: templates/style/css/styles.css

Contains 0 classes and 0 functions.

This file provides shared or global stylesheet rules used across the project. It supports consistent visual formatting.

File: templates/style/css/team_detail.css

Contains 0 classes and 0 functions.

This file defines styles for the team detail page. It controls roster, schedule, announcements, and leaderboard appearance.

File: templates/style/css/weather.css

Contains 0 classes and 0 functions.

This file defines styles for weather display components. It controls forecast layout and formatting.

Folder: templates/style/img/*.png

This folder contains no code and only holds images used in the Vulcan Activity Tracker.

File: templates/style/js/challenges.js

Contains 0 classes and 3 functions:

function renderChallenges(containerId, challenges, isParticipant)

This function renders challenge cards into the selected page container. It mainly uses `formatMetricLabel()` and `renderMedals()`.

function formatMetricLabel(metric)

This function converts stored metric names into user-friendly labels. It supports challenge display formatting.

function renderMedals(medals)

This function formats medal information for display on challenge cards. It supports the challenge list interface.

File: templates/style/js/challenge_details.js

Contains 0 classes and 8 functions:

function formatMetricLabel(metric)

This function converts a challenge metric key into a readable label. It supports the challenge detail page.

function formatActivityLabel(activity)

This function converts an activity key into a readable action label. It supports goal text generation.

function getGoalText(challenge)

This function builds the challenge goal sentence shown on the detail page. It mainly uses formatActivityLabel() and formatMetricLabel().

function parseLocalDate(dateStr)

This function converts a stored date string into a local date object. It supports the challenge date calculations.

function getDaysLeftText(startDateStr, endDateStr)

This function calculates whether a challenge has started, ended, or how many days remain. It mainly uses parseLocalDate().

function renderLeaderboard(leaderboard, metricType)

This function renders the challenge leaderboard on the detail page. It mainly uses `formatMetricLabel()`.

function setActionButton(challenge)

This function chooses whether the action button should join, leave, or delete the challenge. It supports the challenge detail user interface.

async function handleActionClick()

This function sends the selected challenge action to the backend and refreshes the page. It supports join, leave, and delete actions.

File: templates/style/js/clubs.js

Contains 0 classes and 1 function:

function renderClubs(containerId, clubs, isMember)

This function renders club cards into the page. It supports both the public browse view and the member view.

File: templates/style/js/club_details.js

Contains 0 classes and 12 functions:

function formatSport(value)

This function converts a stored sport key into a readable sport label. It supports the club detail page.

function usesDistanceRanking(sportType)

This function decides whether the club leaderboard should rank by distance. It supports leaderboard formatting.

function formatDuration(minutes)

This function converts a number of minutes into an hours-minutes-seconds string. It supports leaderboard and activity display.

function loadActivityLikeCount(username, activityId, countElement)

This function loads the thumbs-up count for a club activity entry. It supports the recent activity tab.

async function loadClub()

This function loads the full club detail data from the backend. It supports the main club detail page.

function setupActionButton(club)

This function configures the main club action button based on ownership, membership, and privacy state. It supports join and request workflows.

async function handleAction()

This function sends the selected club action to the backend and reloads the club data. It supports join, leave, request, cancel, and delete actions.

function renderActiveTab()

This function switches between the members, leaderboard, and recent activity tabs. It controls the visible club detail content.

function renderMembers()

This function renders the club members tab. It displays usernames in a formatted list.

function renderLeaderboard()

This function renders the leaderboard section of the club page. It mainly uses `renderLeaderboardTable()`.

function renderLeaderboardTable(rows, distanceMode)

This function builds the table markup for one leaderboard section. It supports both distance-based and time-based club rankings.

function renderRecentActivity()

This function renders recent activity cards for the club. It mainly uses `loadActivityLikeCount()` and `thumbsUp()`.

File: templates/style/js/dash.js

Contains 0 classes and 14 functions:

function fillDashFriends(currentUser)

This function loads the friend list for the dashboard. It mainly works with `populateDashActivity()`, `populateDashClubs()`, and `populateDashChallenge()`.

function addFriend(selectedUser)

This function sends a request to add a friend and refreshes dashboard content. It supports the social feature.

function fillDashActivity(username)

This function loads dashboard activity data for the current user. It mainly works with `populateDashActivity()`.

function populateDashActivity(data, username)

This function renders the main activity feed on the dashboard. It supports social activity display.

function openFriendModal(friendData)

This function opens the dashboard modal for a selected friend. It supports expanded friend interaction.

function fillFriendActivity(friendUsername)

This function loads activity data for a selected friend. It mainly works with `populateFriendActivities()`.

function populateFriendActivities(activities, friendUsername)

This function renders a friend's activity feed. It supports the dashboard friend modal.

function fillFriendsClub(friendUsername)

This function loads club data for a selected friend. It mainly works with `populateFriendClubs()`.

function populateFriendClubs(clubs)

This function renders club data for a selected friend. It supports the friend modal club view.

function dashLikes(username)

This function loads or refreshes like information for dashboard friend cards. It supports the social feedback feature.

function dashActivityLikes(username, activity_id)

This function loads or refreshes thumbs-up information for dashboard activities. It supports activity reactions.

function fillDashClub(username)

This function loads dashboard club data for the current user. It mainly works with `populateDashClubs()`.

function populateDashClubs(clubs)

This function renders dashboard club cards. It supports quick club access on the dashboard.

function fillDashChallenge()

This function loads dashboard challenge data. It mainly works with populateDashChallenge().

function populateDashChallenge(challenge)

This function renders the dashboard challenge section. It supports challenge visibility on the dashboard.

File: templates/style/js/inbox.js

Contains 0 classes and 6 functions:

function loadTeamInvites()

This function loads pending team invitations into the inbox. It supports the inbox invitation view.

function loadClubRequests()

This function loads pending club join requests into the inbox. It supports owner moderation.

function handleAcceptTeam(teamId)

This function accepts a team invite and reloads the inbox data. It supports the invitation workflow.

function handleDeclineTeam(teamId)

This function declines a team invite and reloads the inbox data. It supports the invitation workflow.

function handleAcceptClub(clubId, requestingUsername)

This function accepts a pending club request and refreshes the inbox. It supports club owner approval.

function handleDeclineClub(clubId, requestingUsername)

This function declines a pending club request and refreshes the inbox. It supports club owner moderation.

File: templates/style/js/info.md

Contains 0 classes and 0 functions:

This file is a Markdown support or note file stored with the JavaScript assets.

File: templates/style/js/leaderboard.js

Contains 0 classes and 21 functions:

async function getLeaderboardData(leaderboardFilter)

This function loads leaderboard data for the selected sport. It supports the public leaderboard page.

function sortTableBySport(sport)

This function updates the leaderboard headers for the selected sport. It supports sport-specific display formatting.

function render(rows)

This function renders the leaderboard rows into the table body. It supports the leaderboard page interface.

function sortAndRender()

This function sorts the current leaderboard data and redraws the table. It supports filtering and sorting.

function normalizeSport(s)

This function converts sport aliases into one standard value. It supports consistent leaderboard calculations.

function parseActivities(row)

This function parses stored activity JSON into usable activity objects. It supports score calculations.

function buildComputedRows(rawRows, sport)

This function builds rows with computed totals and scores for sorting. It supports leaderboard ranking.

function filterBySport(activities, sport)

This function filters activity entries to a selected sport. It supports leaderboard views.

function totalDuration(activities)

This function totals activity duration values. It supports score and summary calculations.

function totalDistance(activities)

This function totals activity distance values. It supports distance-based sports.

function totalSteps(activities)

This function totals step counts. It supports walking leaderboard calculations.

function totalSets(activities)

This function totals set counts. It supports lifting leaderboard calculations.

function yogaStats(activities)

This function calculates total yoga intensity. It supports yoga leaderboard scoring.

function soccerStats(activities)

This function calculates total goals and assists. It supports soccer leaderboard scoring.

function baseballStats(activities)

This function calculates total hits and runs. It supports baseball leaderboard scoring.

function footballStats(activities)

This function calculates total touchdowns. It supports football leaderboard scoring.

function volleyballStats(activities)

This function calculates total kills. It supports volleyball leaderboard scoring.

function basketballStats(activities)

This function calculates total points, rebounds, and assists. It supports basketball leaderboard scoring.

function hockeyStats(activities)

This function calculates total goals and assists. It supports hockey leaderboard scoring.

function computeScore(row, sport)

This function chooses the correct scoring rule for the selected sport. It supports leaderboard ranking.

function init()

This function loads the initial leaderboard view when the page opens. It prepares the first rendered table.

File: templates/style/js/like.js

Contains 0 classes and 2 functions:

function likeFeature(username, friendName)

This function starts the friend like workflow. It supports social interaction on the dashboard.

function thumbsUp(username, friendName, activity_id, modallikeBtn, modallikeCount)

This function toggles a thumbs-up on an activity and updates the displayed count. It supports activity reactions.

File: templates/style/js/login.js

Contains 0 classes and 4 functions:

function validateEmail(email)

This function checks whether an email address is in a valid format. It supports frontend validation before submission.

function hashPassword(password)

This function hashes a password before it is sent. It supports safer login and account workflows.

function sendLoginData(email, username, hashPassword)

This function sends login or registration data to the backend. It supports authentication workflows.

function backToLogin()

This function returns the user to the login page view. It supports page navigation.

File: templates/style/js/logout.js

Contains 0 classes and 1 function:

function logout()

This function calls the logout process and clears the current session state in the frontend. It supports signing out.

File: templates/style/js/maps.js

Contains 0 classes and 6 functions:

function calculateDistanceMiles(coords)

This function calculates total route distance from route coordinates. It supports saved route summaries.

function storeRoutes(routePoints, route_name)

This function sends a new route to backend storage. It supports the map save workflow.

function loadUserRoutes()

This function loads all saved routes for the current user. It supports the maps page startup.

function displayMaps(routes)

This function renders saved routes into the map interface. It supports route browsing and selection.

function deleteRoute(li, routeName)

This function removes a route from the page and begins deletion. It supports route management.

function deleteRouteFromStorage(routeName)

This function sends the backend request to delete a saved route. It supports permanent route removal.

File: templates/style/js/modal.js

Contains 0 classes and 0 named functions:

This file uses inline click handlers instead of named functions. It opens and closes the modal window based on button, close icon, and background clicks.

File: templates/style/js/resetPassword.js

Contains 0 classes and 2 functions:

function backToLogin()

This function returns the user to the login page from the reset password view. It supports page navigation.

function resetPassword()

This function sends the new password to the backend reset workflow. It supports password recovery.

File: templates/style/js/settings.js

Contains 0 classes and 4 functions:

function fillQuickStats()

This function loads quick statistics for the settings page. It mainly works with populateQuick().

function loadUserInfo()

This function loads user profile information for display on the settings page. It mainly works with fillDisplayInfo().

function populateQuick(data)

This function renders quick statistic values into the settings page. It supports settings display.

function fillDisplayInfo(data)

This function renders user display information such as name and email. It supports the settings interface.

File: templates/style/js/teams.js

Contains 0 classes and 2 functions:

async function fetchAllTeams()

This function loads all teams from the backend. It supports the teams landing page.

function renderTeams()

This function filters and renders the visible team cards. It supports gender-based team browsing.

File: templates/style/js/team_detail.js

Contains 0 classes and 29 functions:

async function loadTeam()

This function loads the main team detail data and renders the roster. It supports the team detail page.

async function loadAnnouncements()

This function loads team announcements from the backend. It supports the announcements section.

async function loadSchedule()

This function loads schedule events for the team. It supports the schedule section.

async function postAnnouncement()

This function submits a new team announcement and refreshes the list. It supports coach tools.

async function createScheduleEvent()

This function submits a new schedule event and refreshes the schedule list. It supports coach tools.

async function sendInvite(invitedUsername)

This function sends a team invitation to another user. It supports coach roster management.

function formatDate(dateString)

This function converts stored date values into readable date text. It supports announcements and schedules.

async function loadTeamLeaderboard()

This function loads the team leaderboard widget. It supports the team detail page.

async function getLeaderboardData(leaderboardFilter)

This function loads leaderboard data for the selected team sport filter. It supports team rankings.

function normalizeSport(s)

This function normalizes sport aliases into one standard value. It supports leaderboard calculations.

function sortTableBySport(sport)

This function updates leaderboard headers for the selected sport. It supports team leaderboard formatting.

function parseActivities(row)

This function parses stored activity JSON into usable activity objects. It supports score calculations.

function filterBySport(activities, sport)

This function filters activity records by sport. It supports leaderboard views.

function totalDuration(activities)

This function totals activity duration values. It supports ranking calculations.

function totalDistance(activities)

This function totals activity distance values. It supports distance-based sports.

function totalSteps(activities)

This function totals step values. It supports walking score calculations.

function totalSets(activities)

This function totals set values. It supports lifting score calculations.

function yogaStats(activities)

This function calculates yoga intensity totals. It supports yoga rankings.

function soccerStats(activities)

This function calculates goals and assists. It supports soccer rankings.

function baseballStats(activities)

This function calculates hits and runs. It supports baseball rankings.

function footballStats(activities)

This function calculates touchdowns. It supports football rankings.

function volleyballStats(activities)

This function calculates kills. It supports volleyball rankings.

function basketballStats(activities)

This function calculates points, rebounds, and assists. It supports basketball rankings.

function hockeyStats(activities)

This function calculates goals and assists. It supports hockey rankings.

function computeScore(row, sport)

This function applies the proper scoring rule for the selected sport. It supports team leaderboard ranking.

function buildComputedRows(rawRows, sport)

This function builds leaderboard rows with computed totals and score values. It supports sorting.

function render(rows)

This function renders team leaderboard rows into the table. It supports the leaderboard interface.

function sortAndRender()

This function sorts the current team leaderboard data and redraws the table. It supports filtering and sorting.

File: templates/style/js/training.js

Contains 0 classes and 4 functions:

function collectSportData(form)

This function collects training form data into a structured activity object. It supports activity submission.

function sendActivityData(data, username)

This function sends activity data to the backend. It supports the training page workflow.

function fillActivityTable(username)

This function loads the current user's activity history table. It supports the training page.

function populateActivityTable(data)

This function renders activity rows into the training table. It supports activity history display.

File: templates/style/js/verifyLogin.js

Contains 0 classes and 3 functions:

function hashPassword(password)

This function hashes a password before verification. It supports secure login handling.

function sendLoginData(username, hashPassword, email)

This function sends verification data to the backend. It supports the login verification flow.

function toDashboard()

This function redirects the user to the dashboard after successful verification. It supports login completion.

File: templates/style/js/weather.js

Contains 0 classes and 1 function:

function loadForecast()

This function loads weather forecast data for the application interface. It supports the weather display feature.